Integration Demo: Building a Schedule Integration that Invokes an App-Driven Integration

Author: Moon Ray Lo, Oracle ACE Apprentice

Date: October 24, 2026

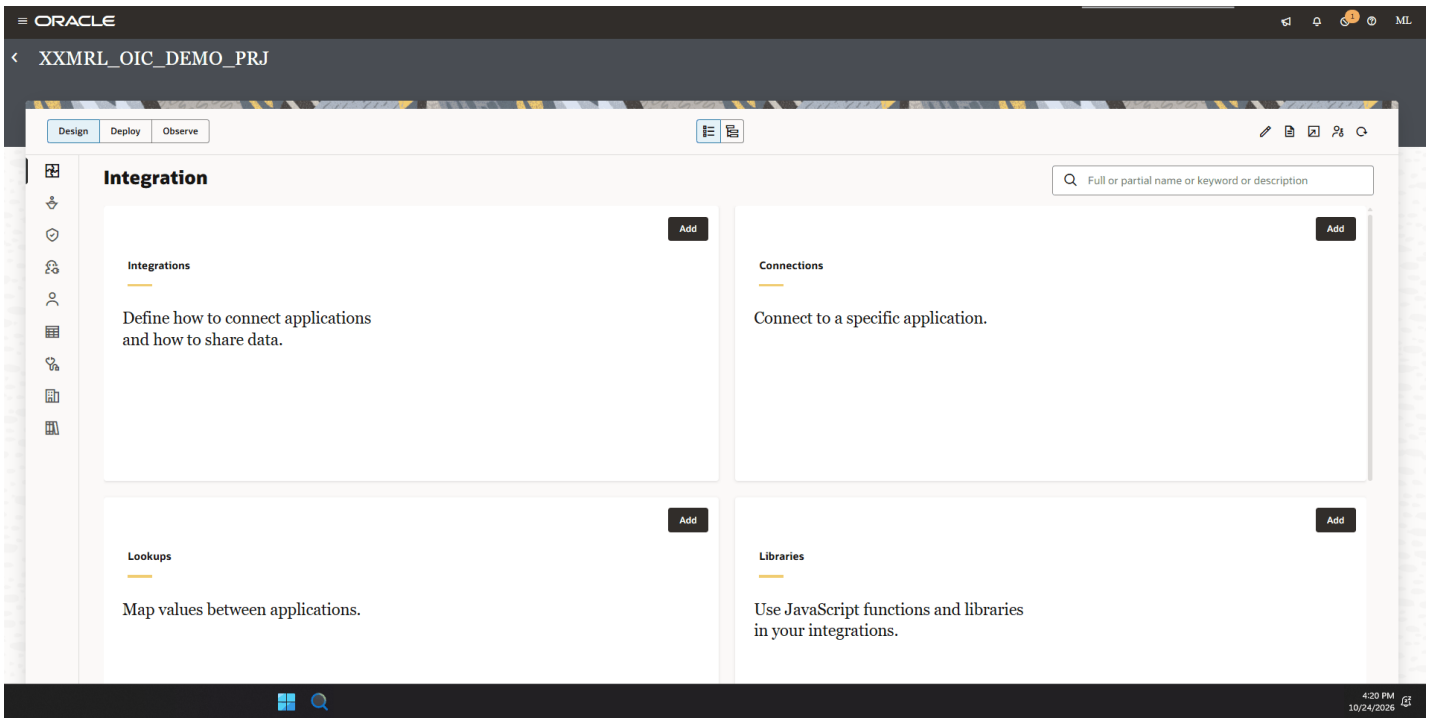
Product: Oracle Integration 3 (version 26.10.0.1), development instance

Project: XXMRL_OIC_DEMO_PRJ

Note: instance host names and user IDs are masked in the screenshots.

Part 1: Create the REST Connection

I opened the project and, in the Connections section, clicked Add:

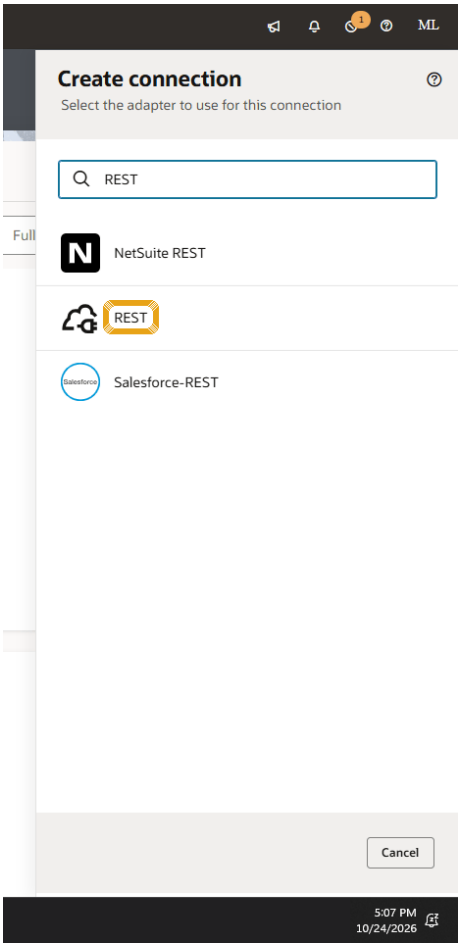


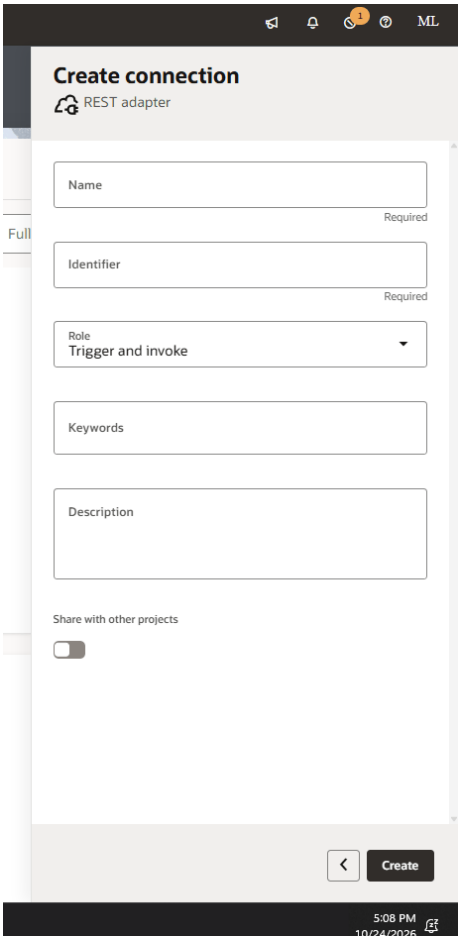
I searched for the REST adapter and created the connection with these details:

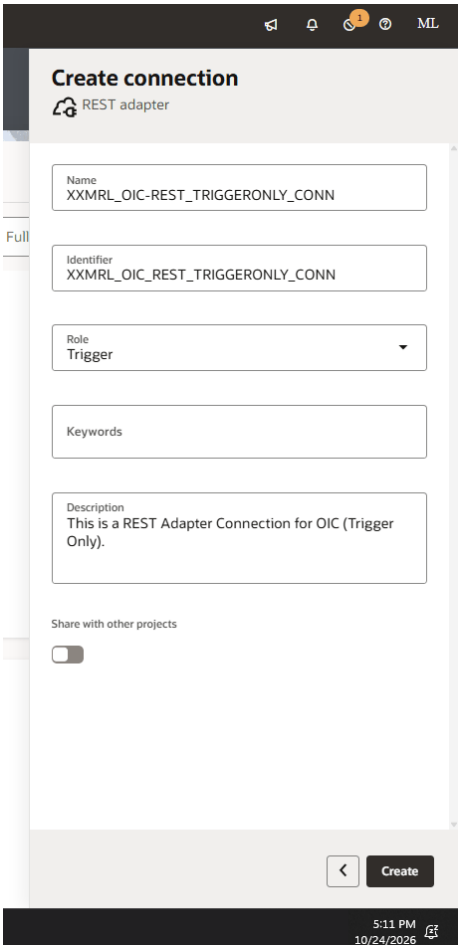
Name: XXMRL_OIC-REST_TRIGGERONLY_CONN

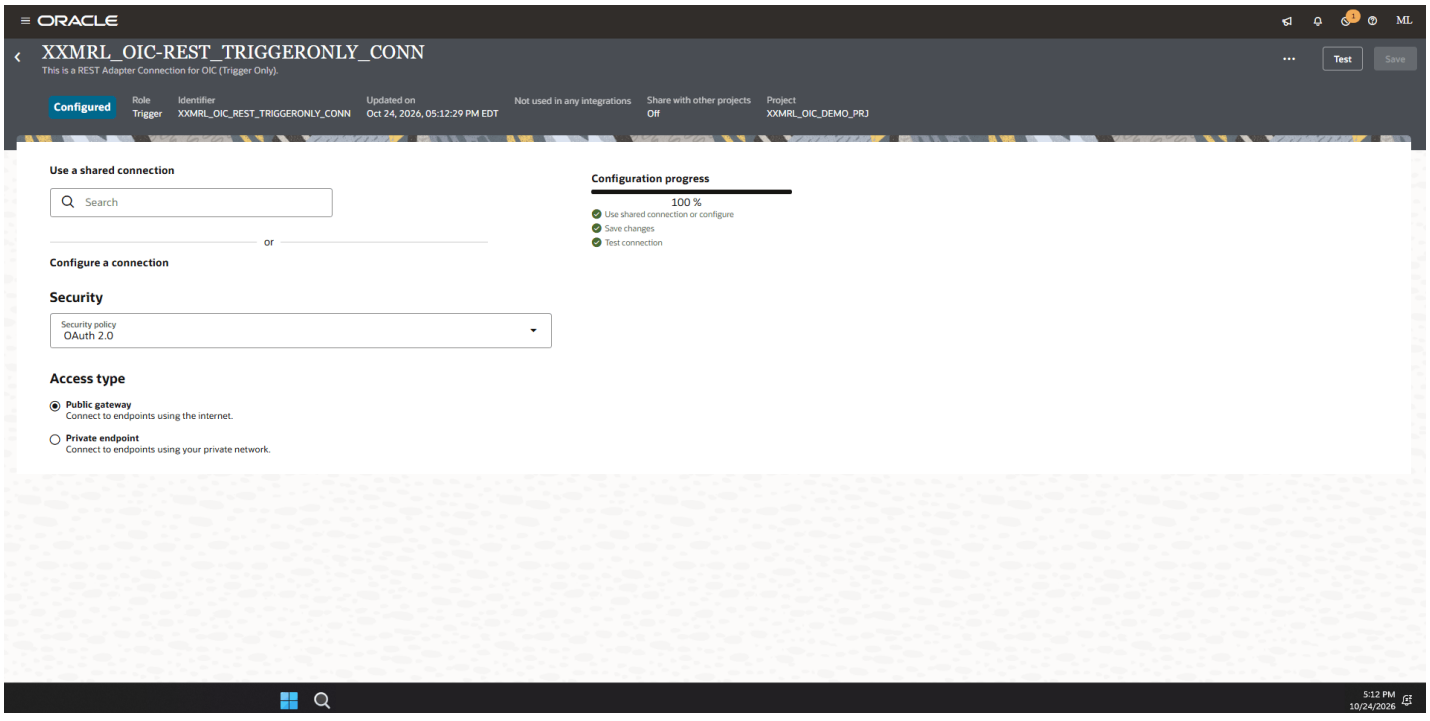
Role: Trigger

Description: This is a REST Adapter Connection for OIC (Trigger Only).

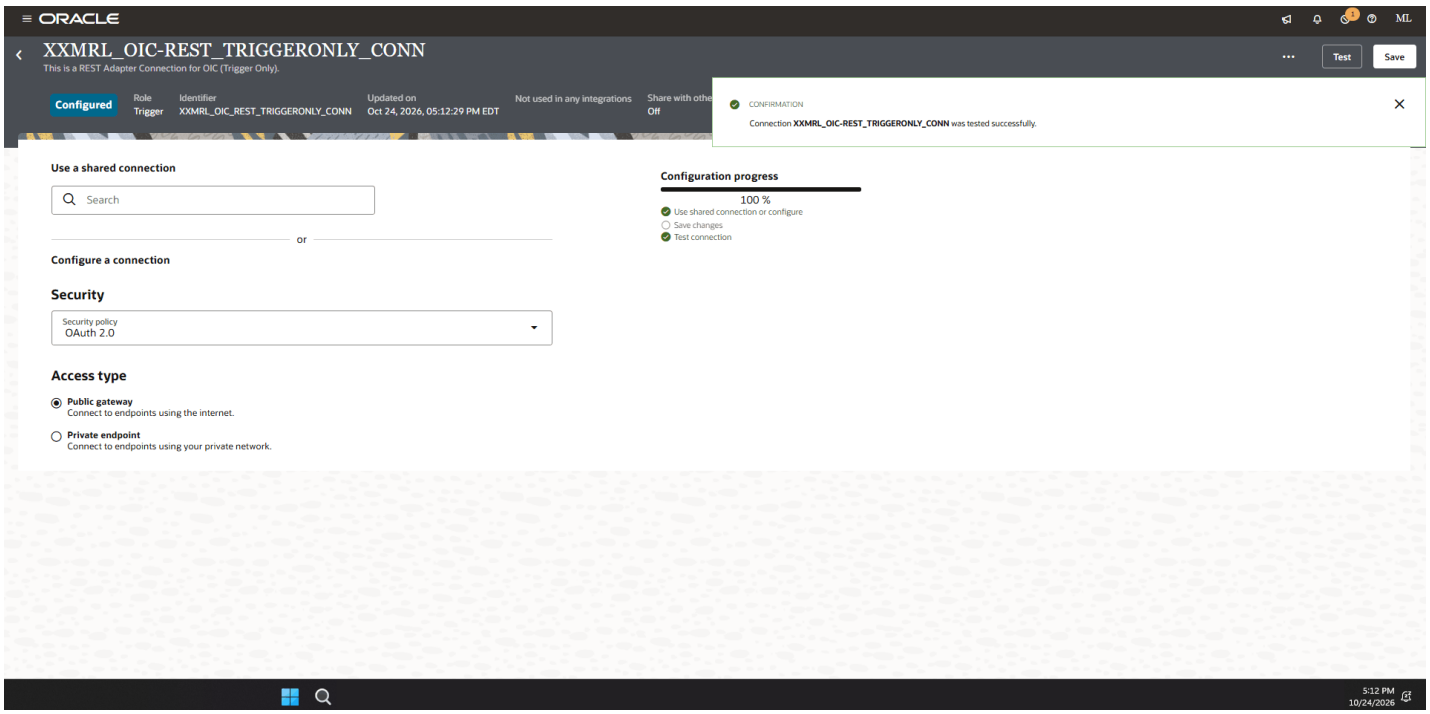




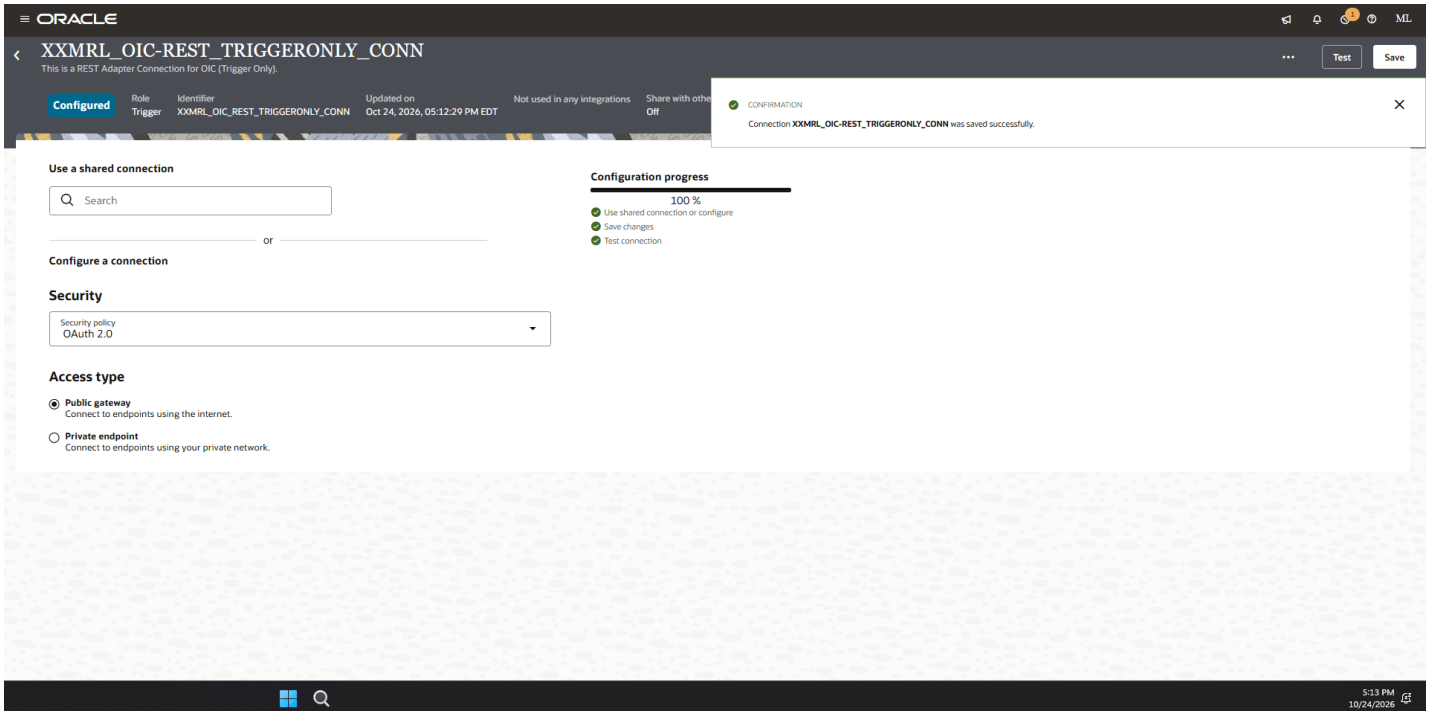


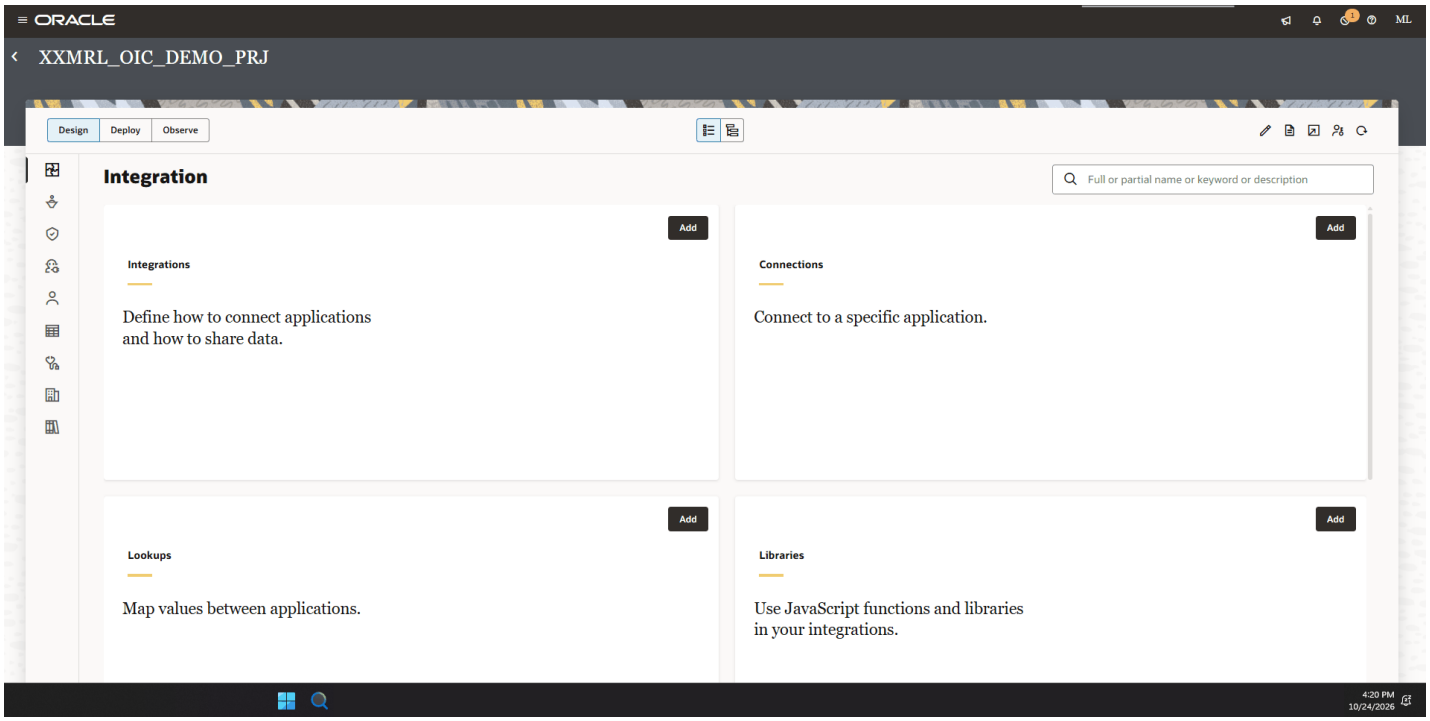


I kept the default security policy (OAuth 2.0) and access type (Public gateway), then clicked Test.



Once the test was successful, I clicked Save to complete the connection setup.



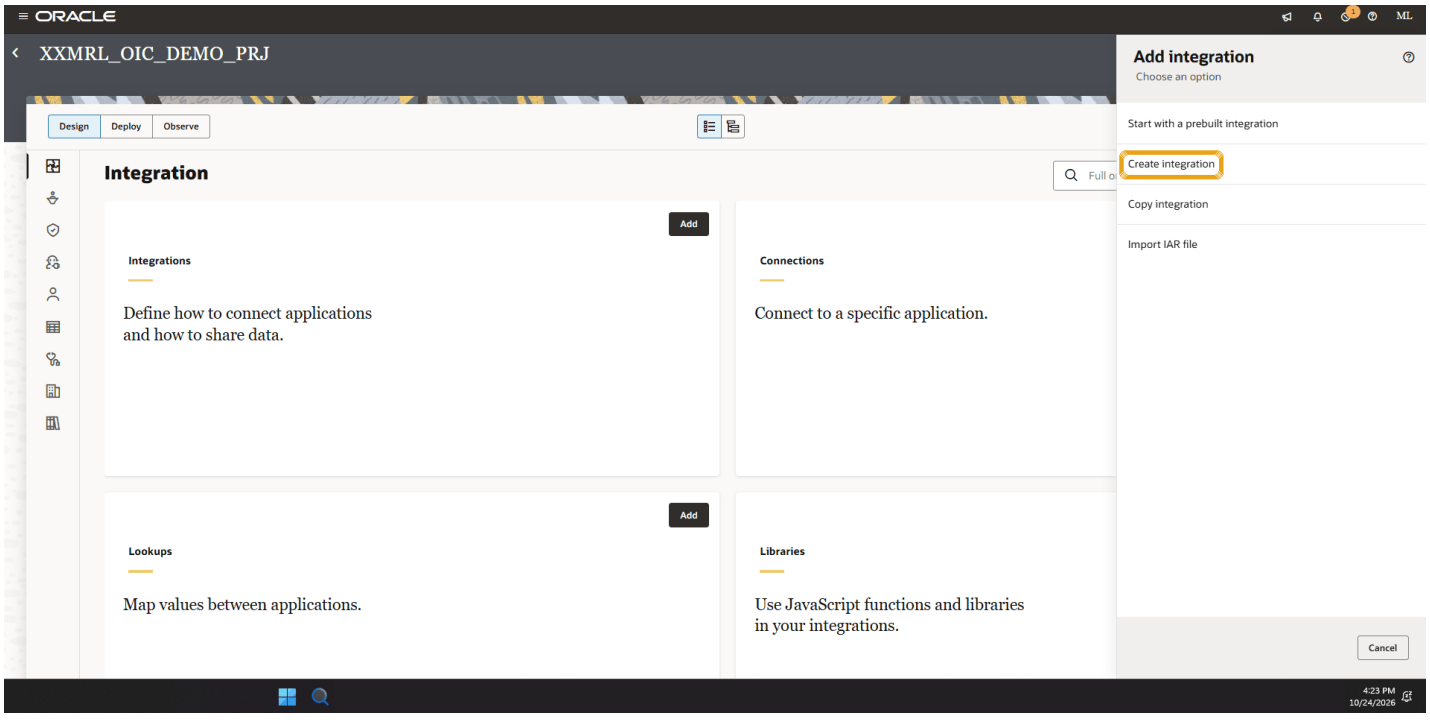


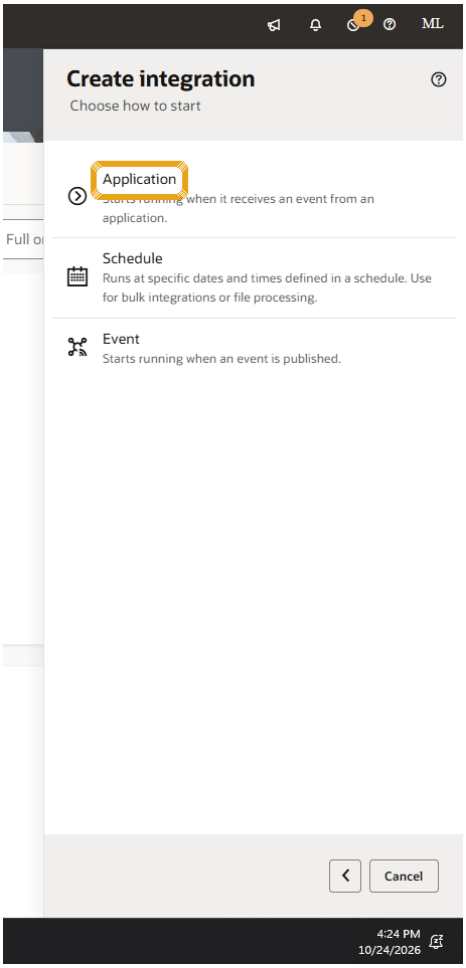
Part 2.1: Create the App-Driven Integration

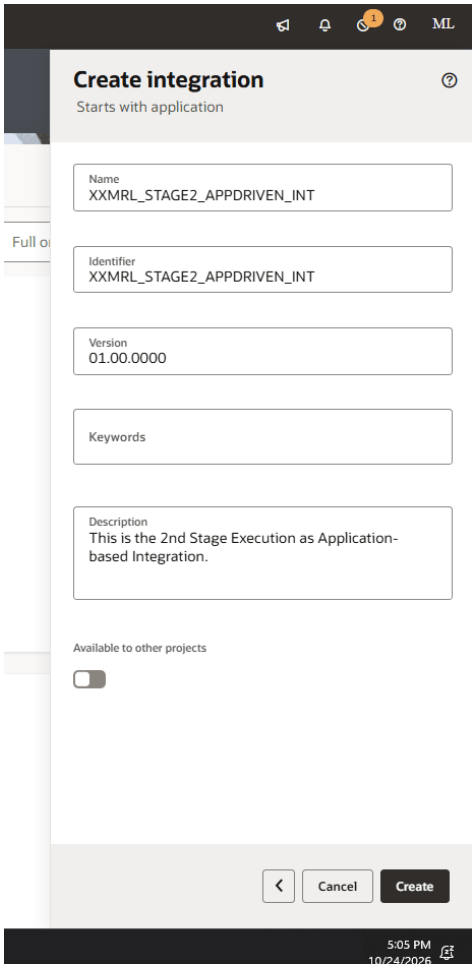
In the project, I went to Integrations, clicked Add, chose Create integration and selected Application:

Name: XXMRL_STAGE2_APPDRIVEN_INT

Description: This is the 2nd Stage Execution as Application-based Integration.

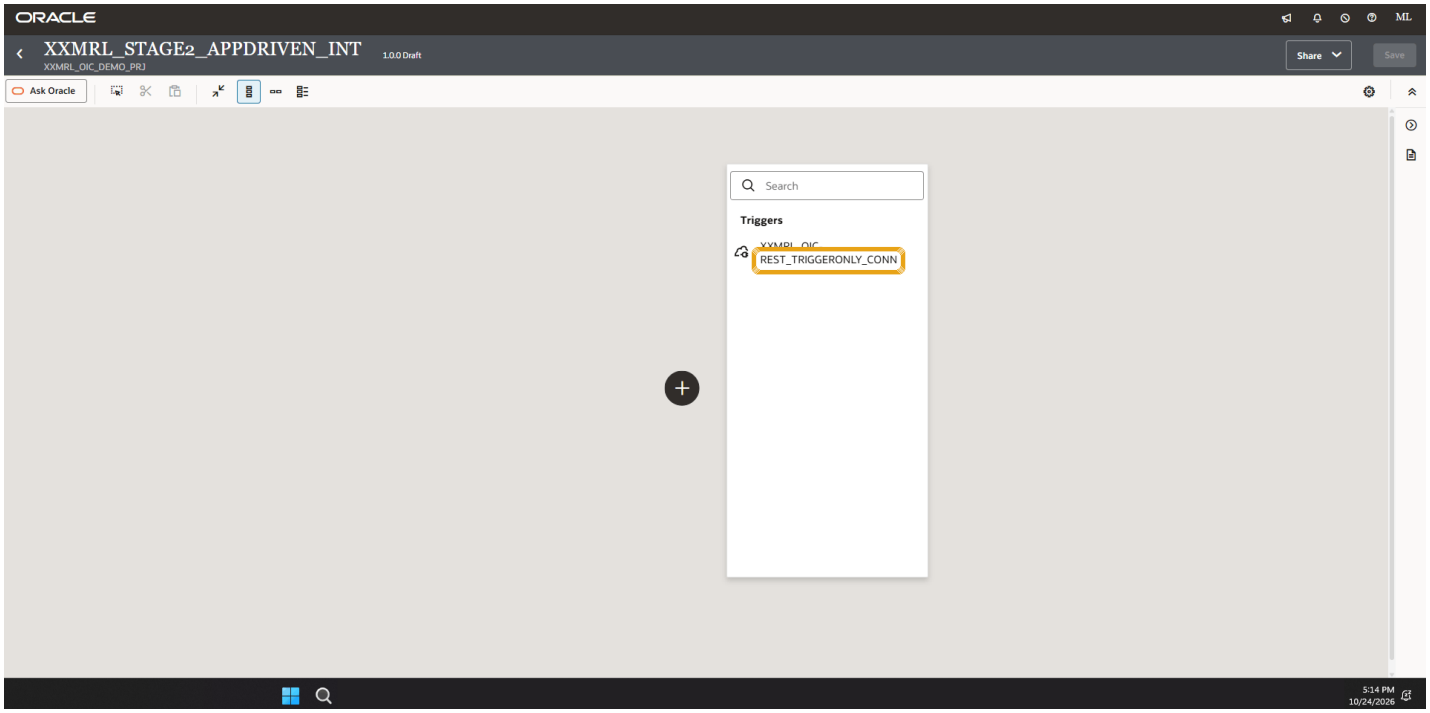


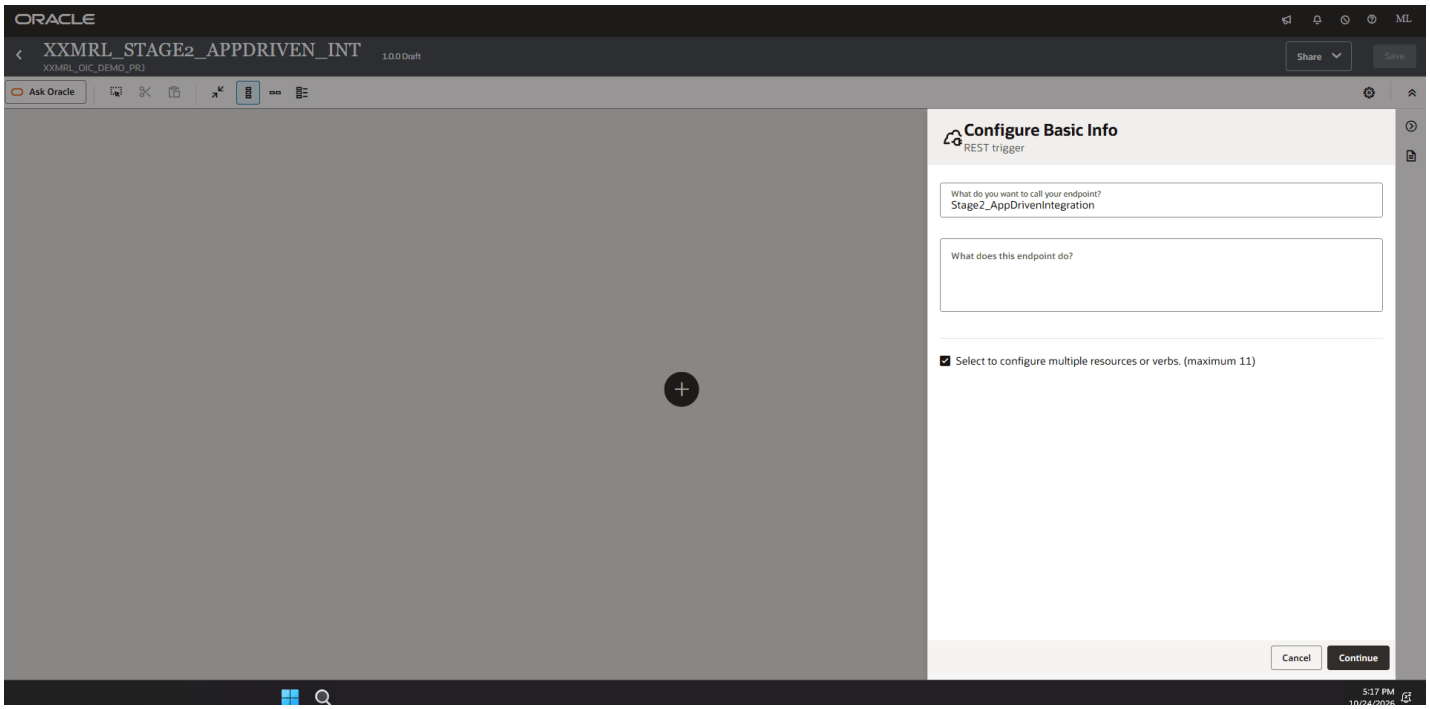




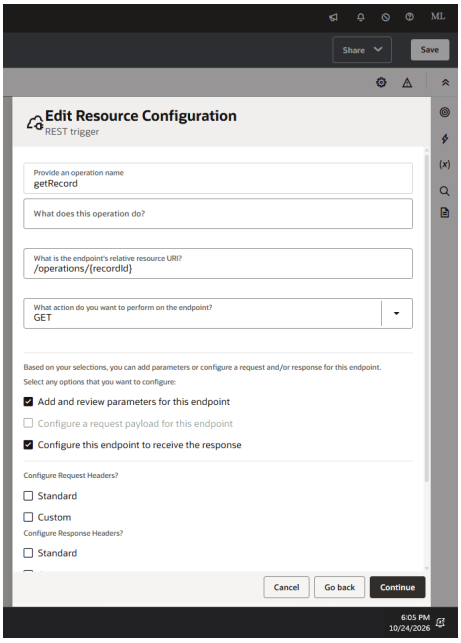
I clicked Create to start building the integration.

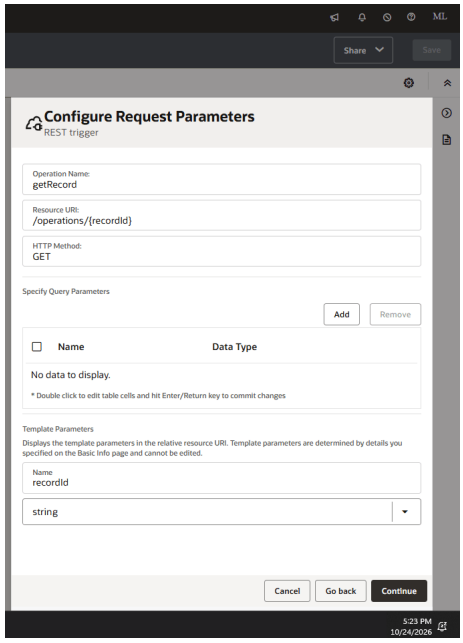
I selected the trigger connection I created in Part 1, named the endpoint, and enabled multiple resources so the integration could have two operations:

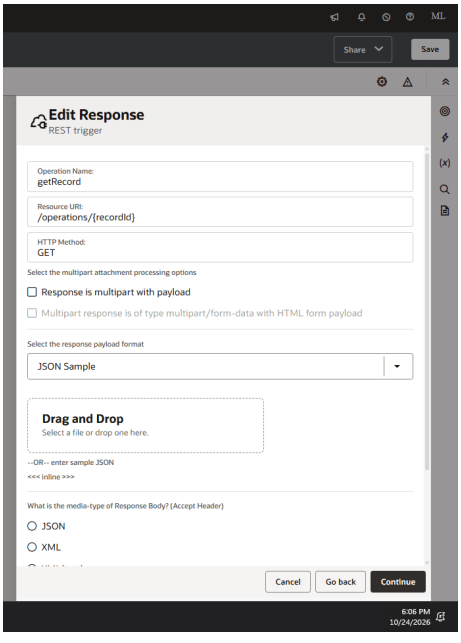




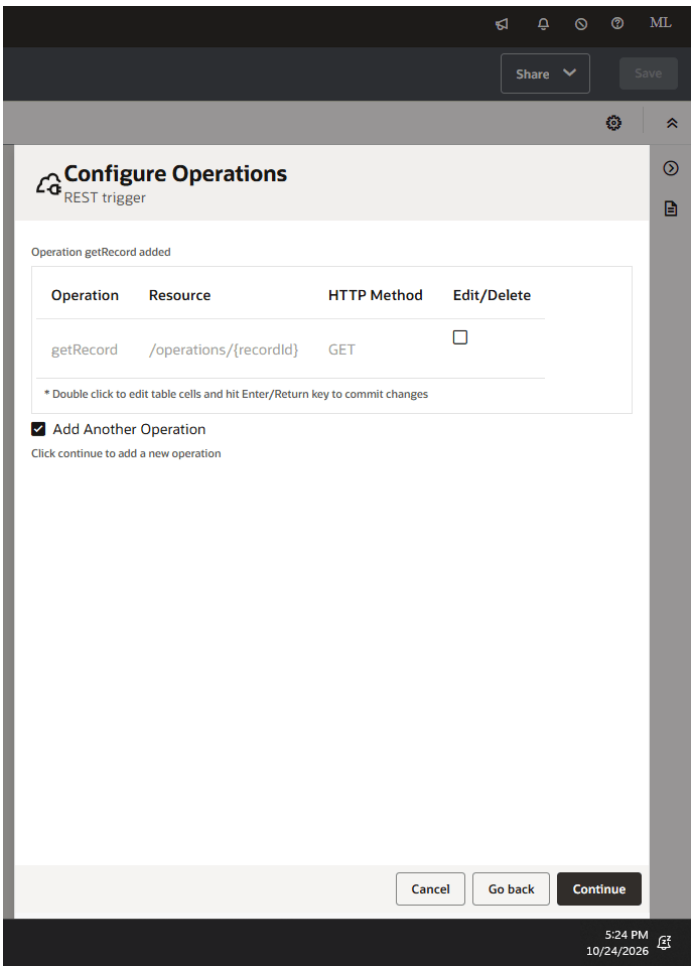
I configured the GET operation (getRecord) with the relative URI /operations/{recordId}, the recordId template parameter, and a JSON sample for the response:



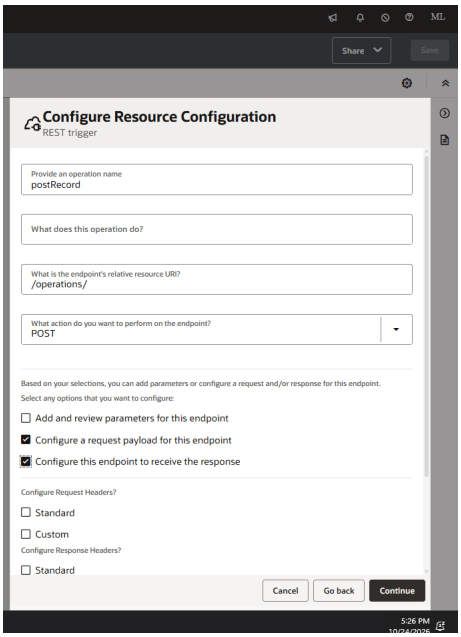


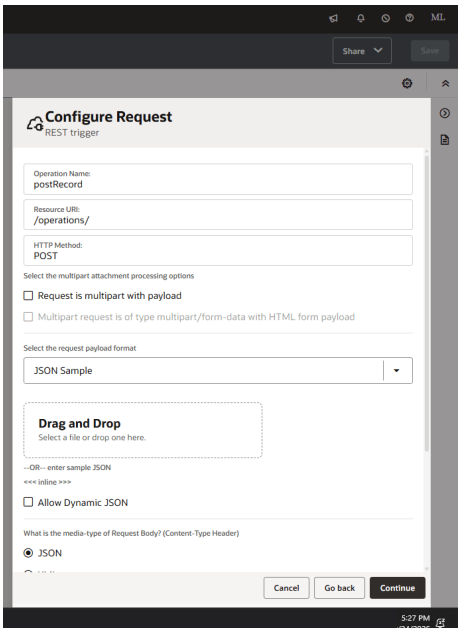


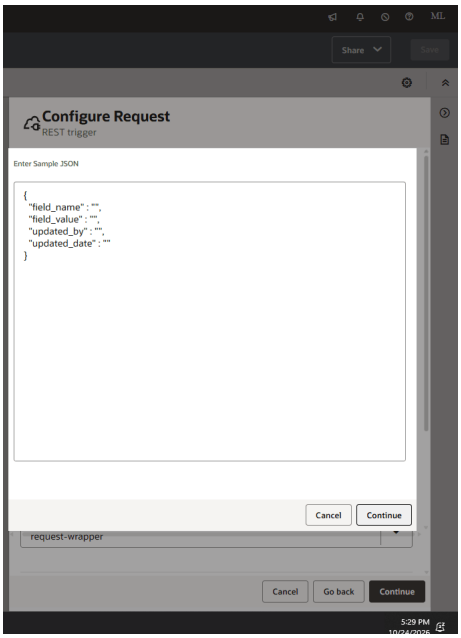




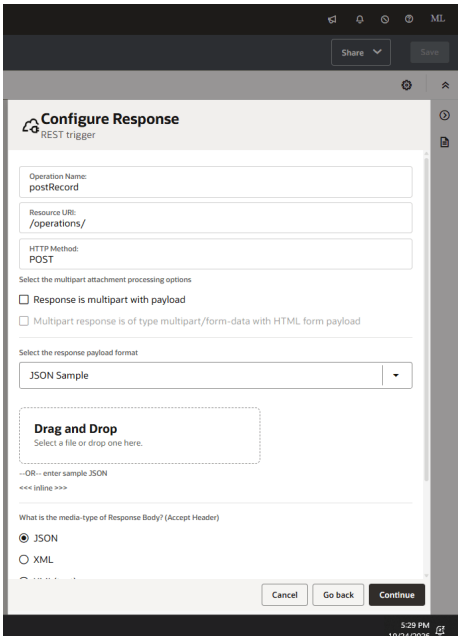
I then added a second operation and configured the POST operation (postRecord) with the relative URI /operations/, plus JSON samples for the request and the response:



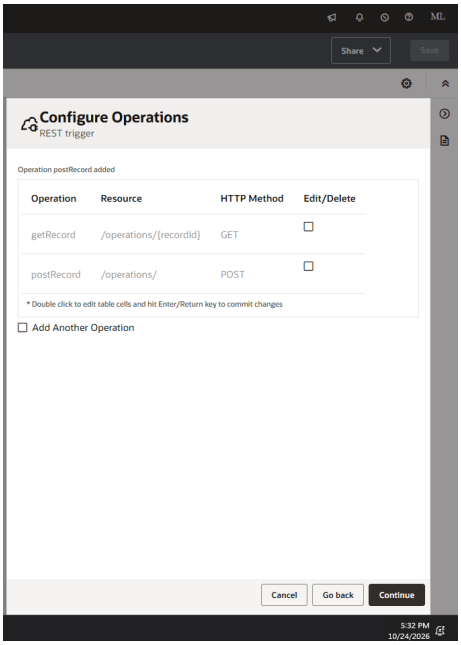




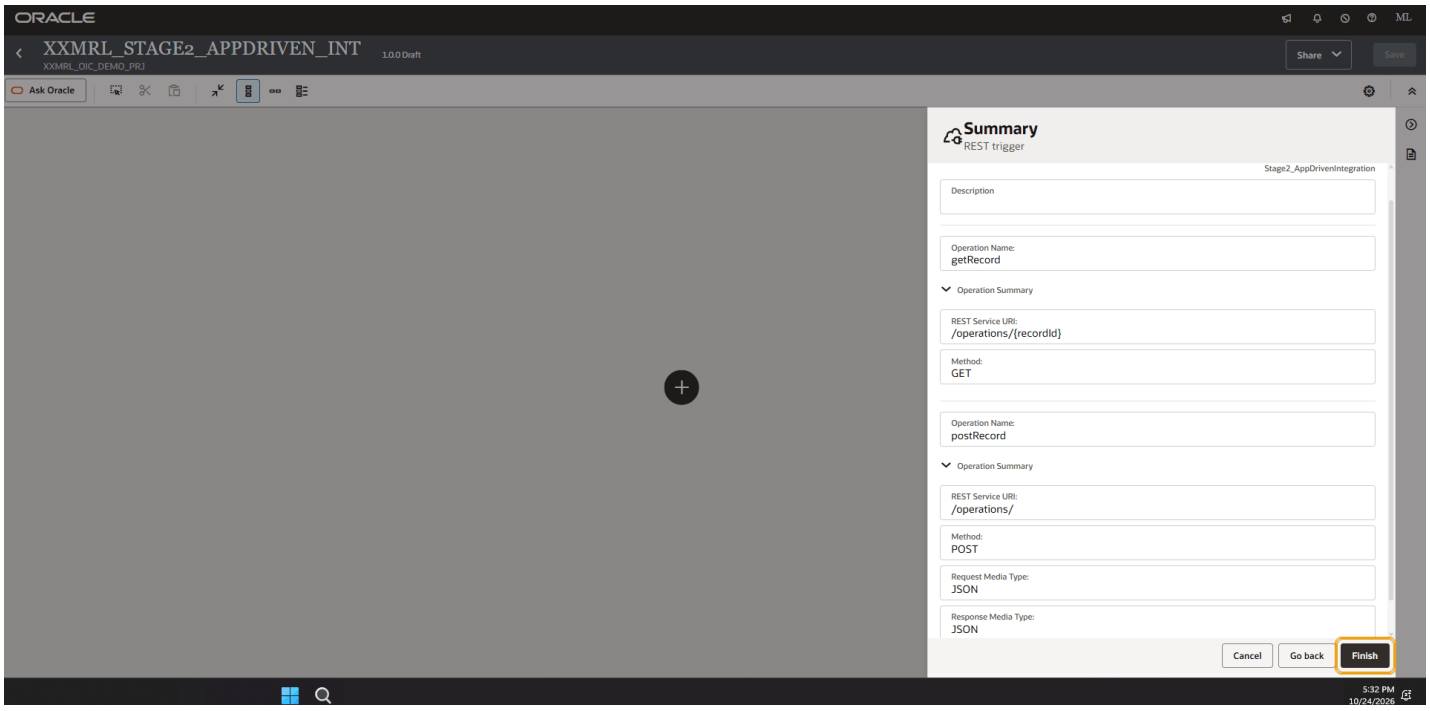
I reviewed the summary and clicked Finish:

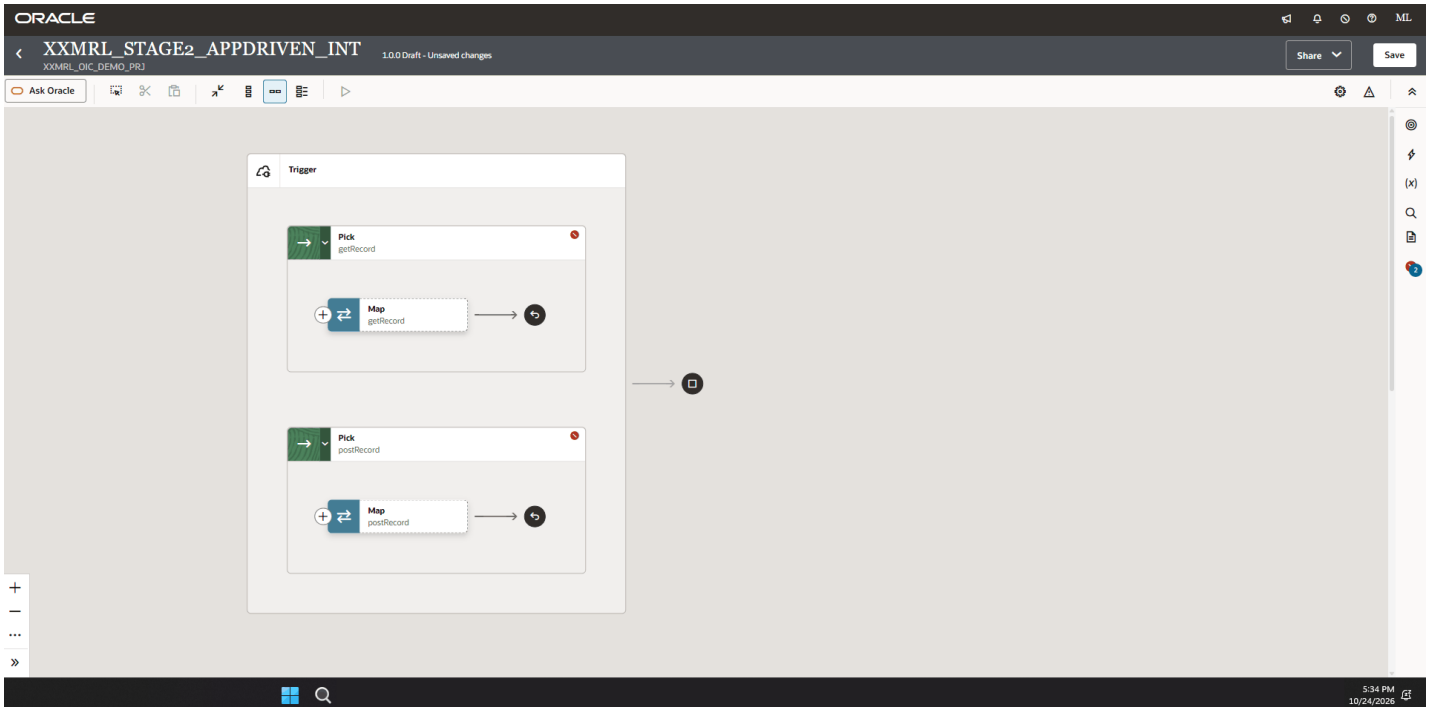






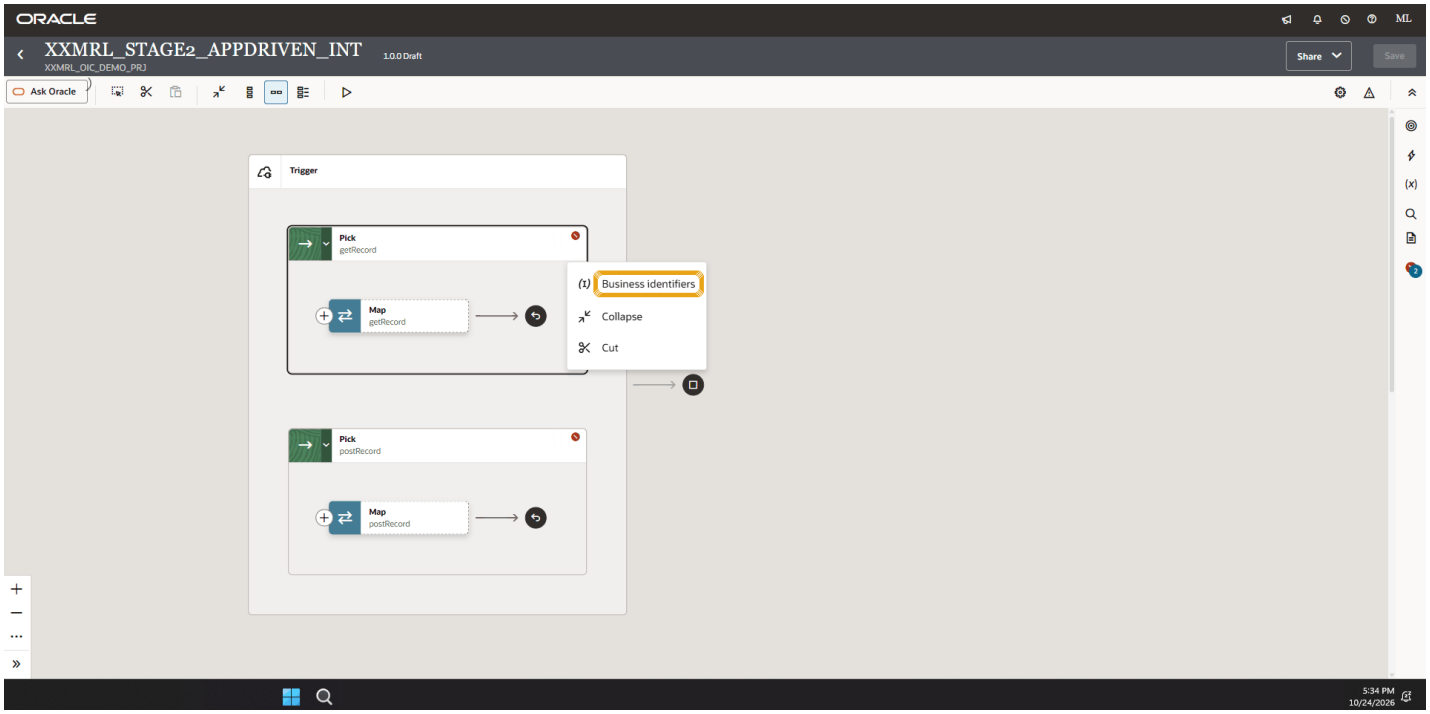
The trigger now had both operations, each with its own response mapper.

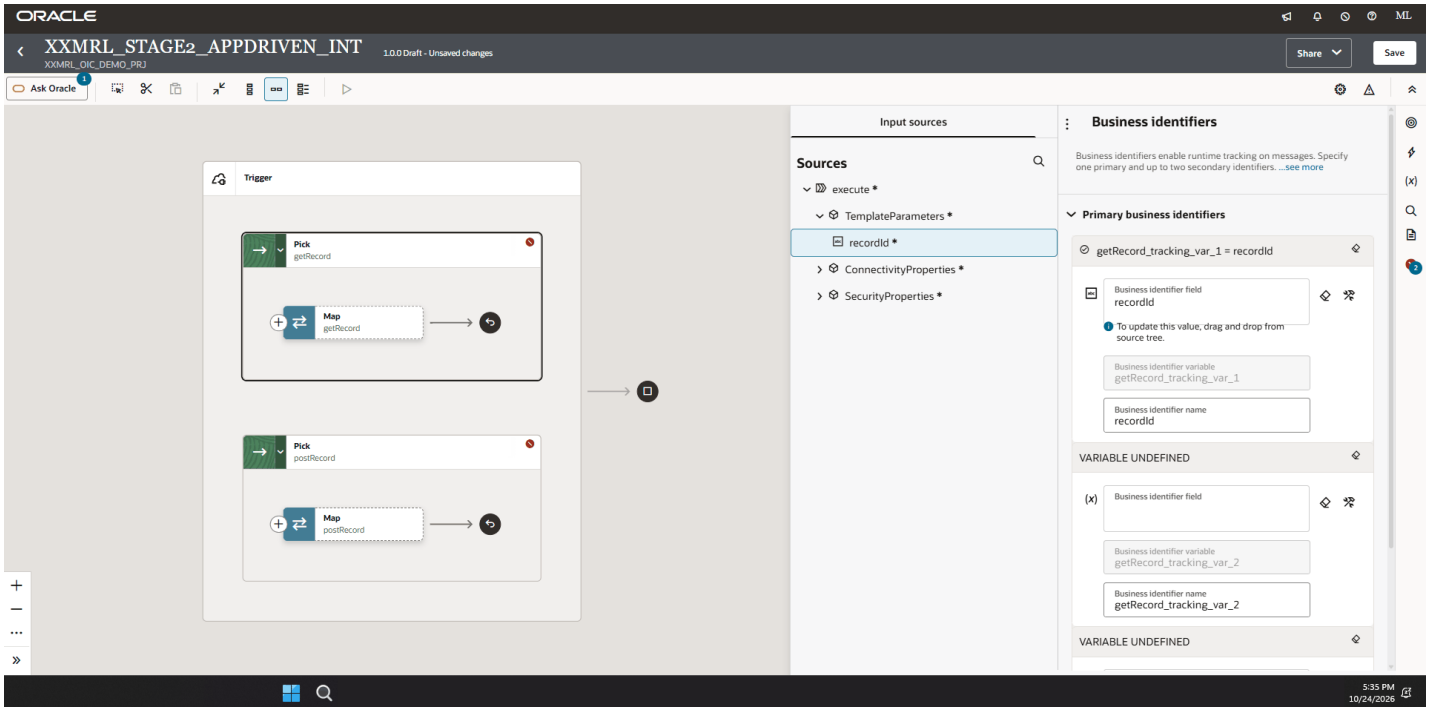




I clicked Save so I wouldn't lose my changes.

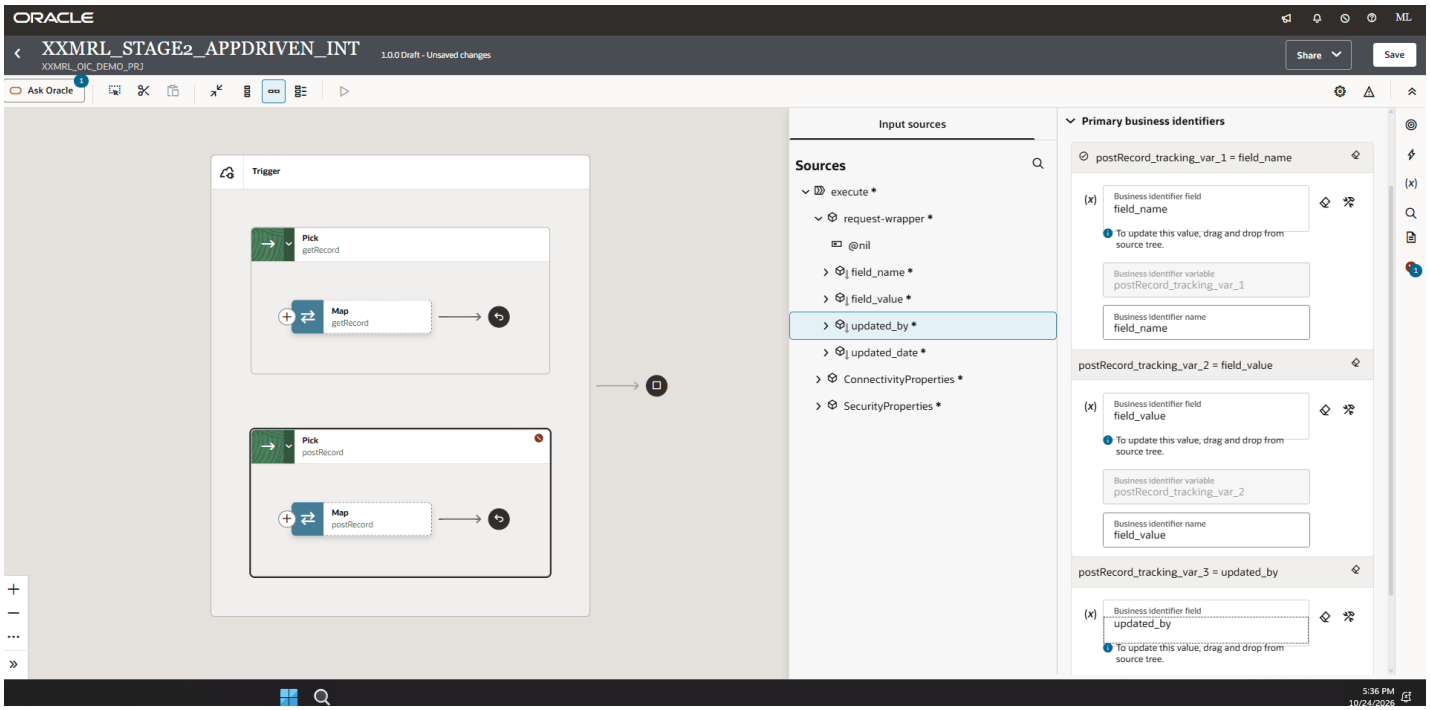
Next, I assigned the business identifiers for each operation:





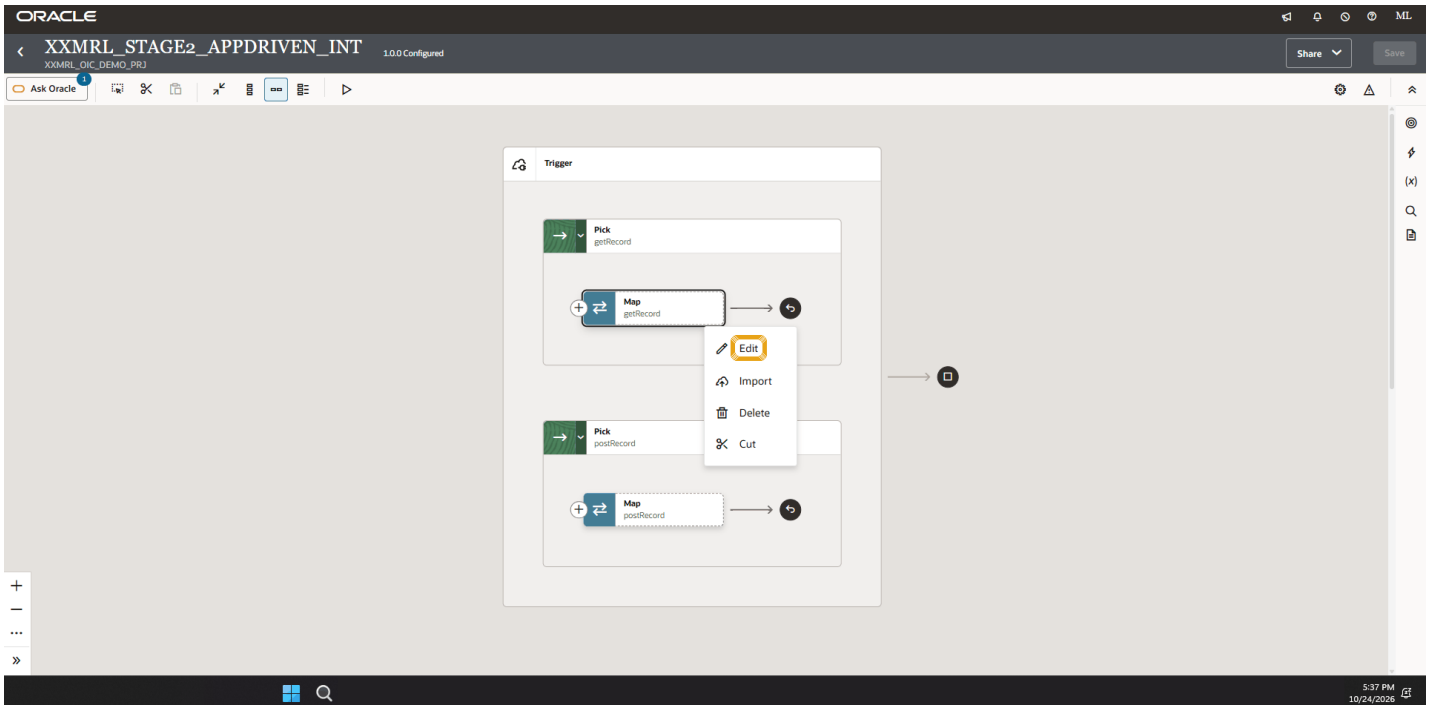
I clicked Save.

I did the same for the other operation:



I clicked Save once done.

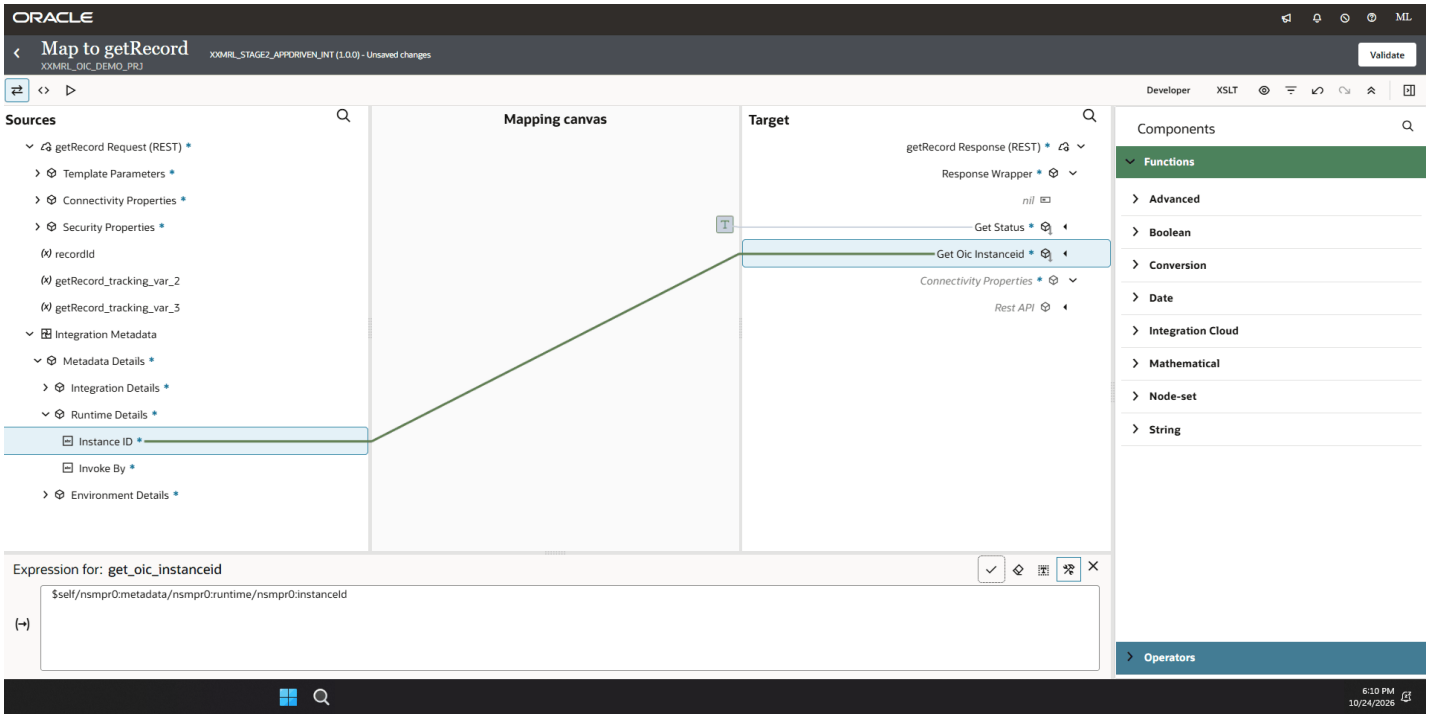
Next, I edited the response mapper for each operation:



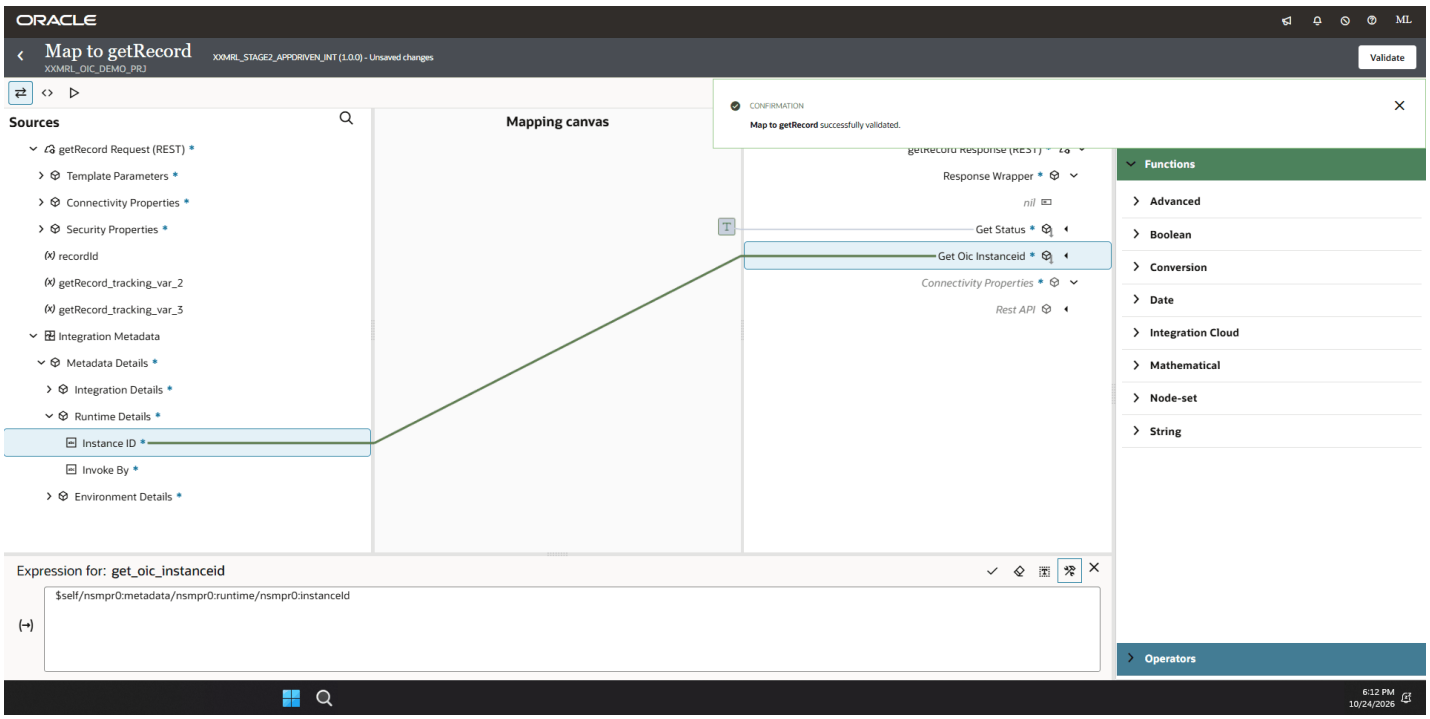
I expanded the response wrapper, right-clicked each field and chose Create Target Node. Then I clicked the tool icon at the bottom to switch to Developer View, and used Toggle Functions to open the functions panel.

For demo purposes, I set get_status to the hardcoded value ‘SUCCESS’ and get_oic_instanceid to the runtime OIC instance ID from the Integration Metadata section.

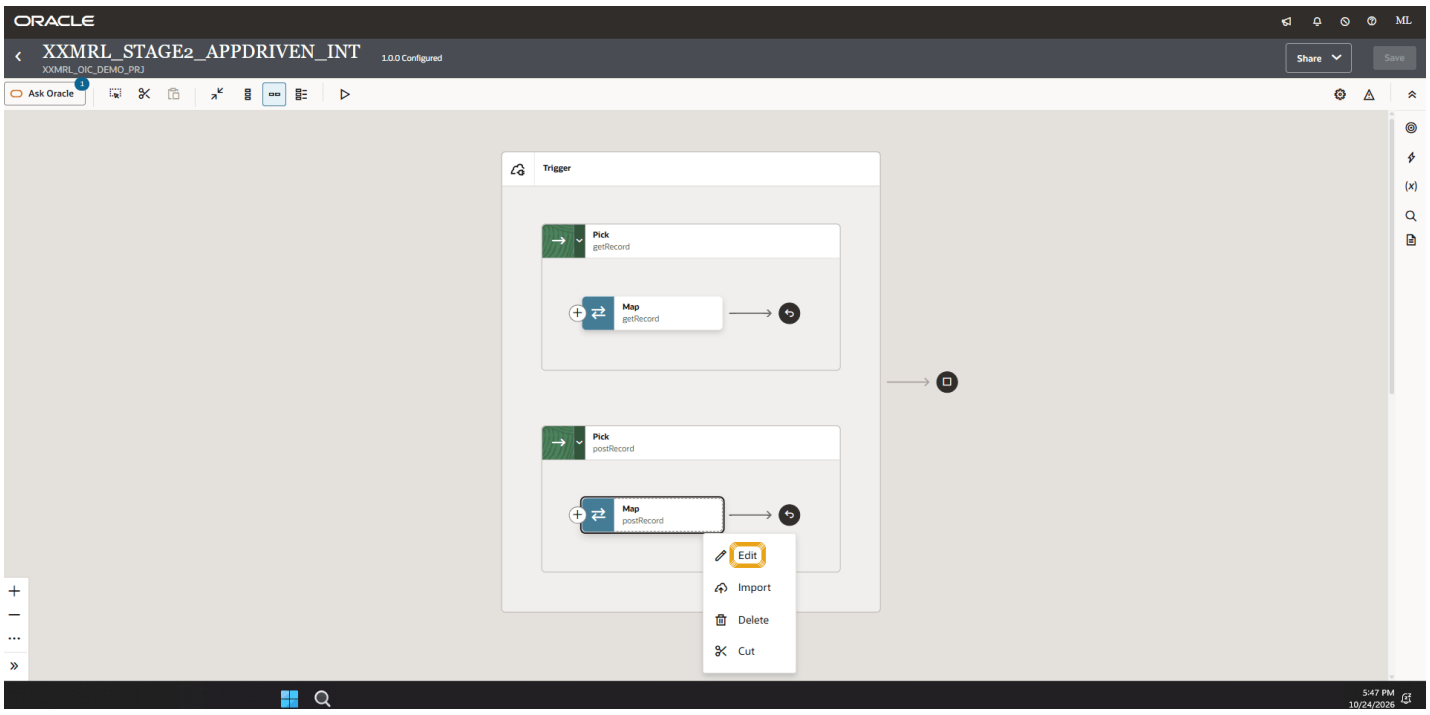
I clicked the check mark to save each field mapping:



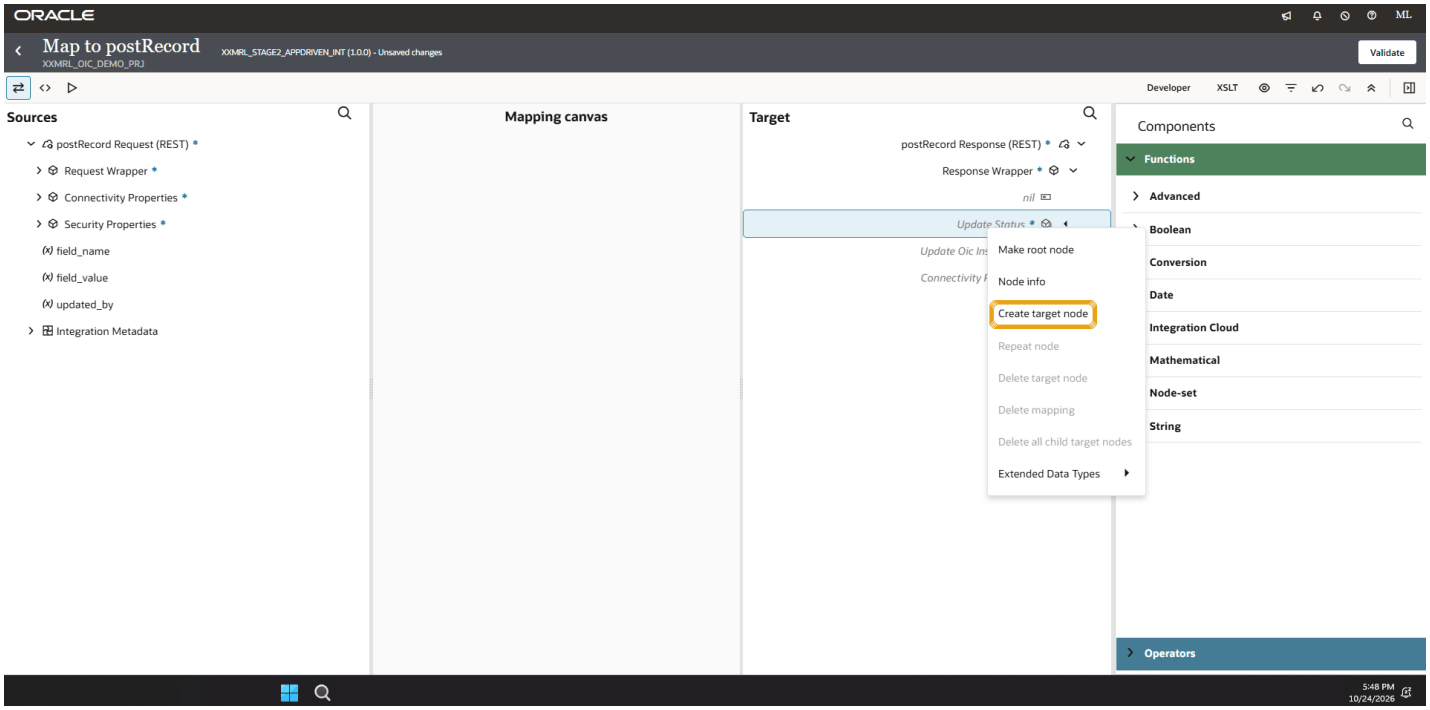
Then I clicked Validate to confirm the mapping:



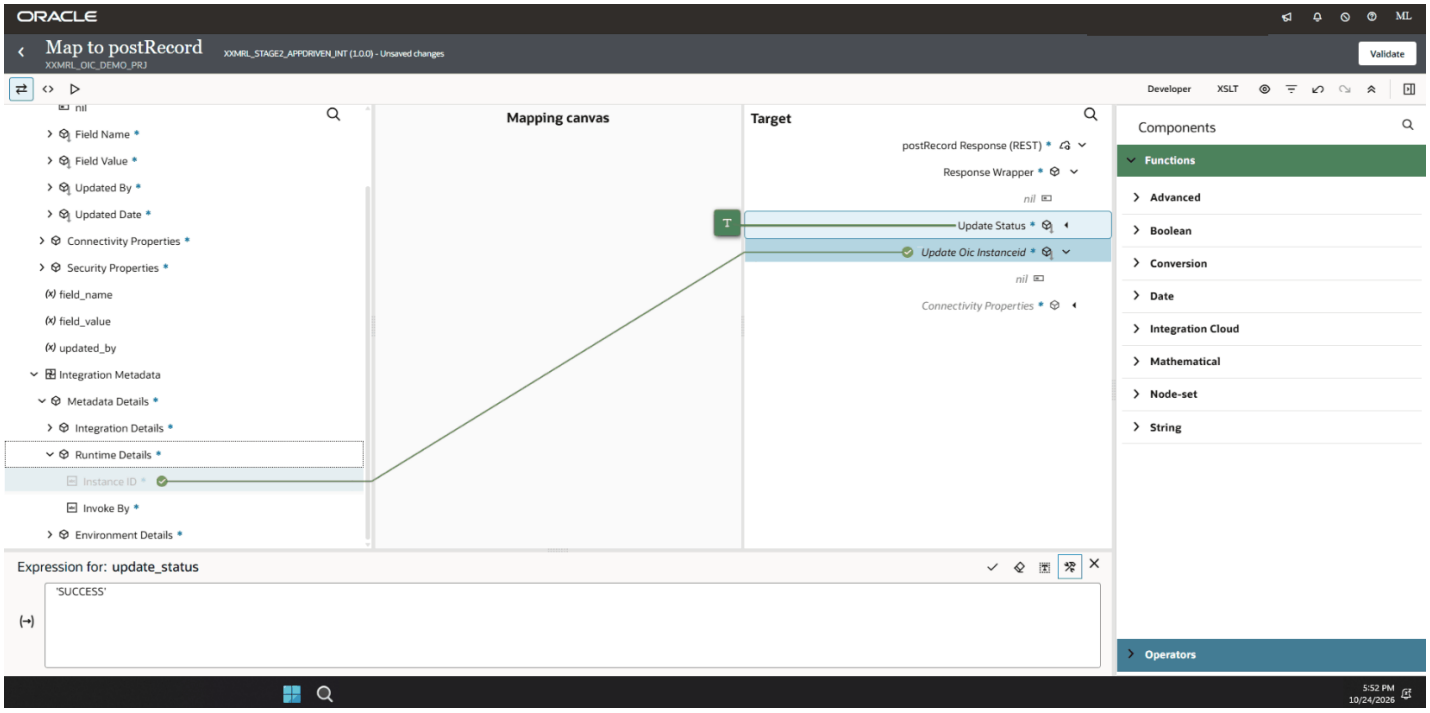
I went back to the main integration orchestration and clicked Save, then opened the response mapper of the other operation:



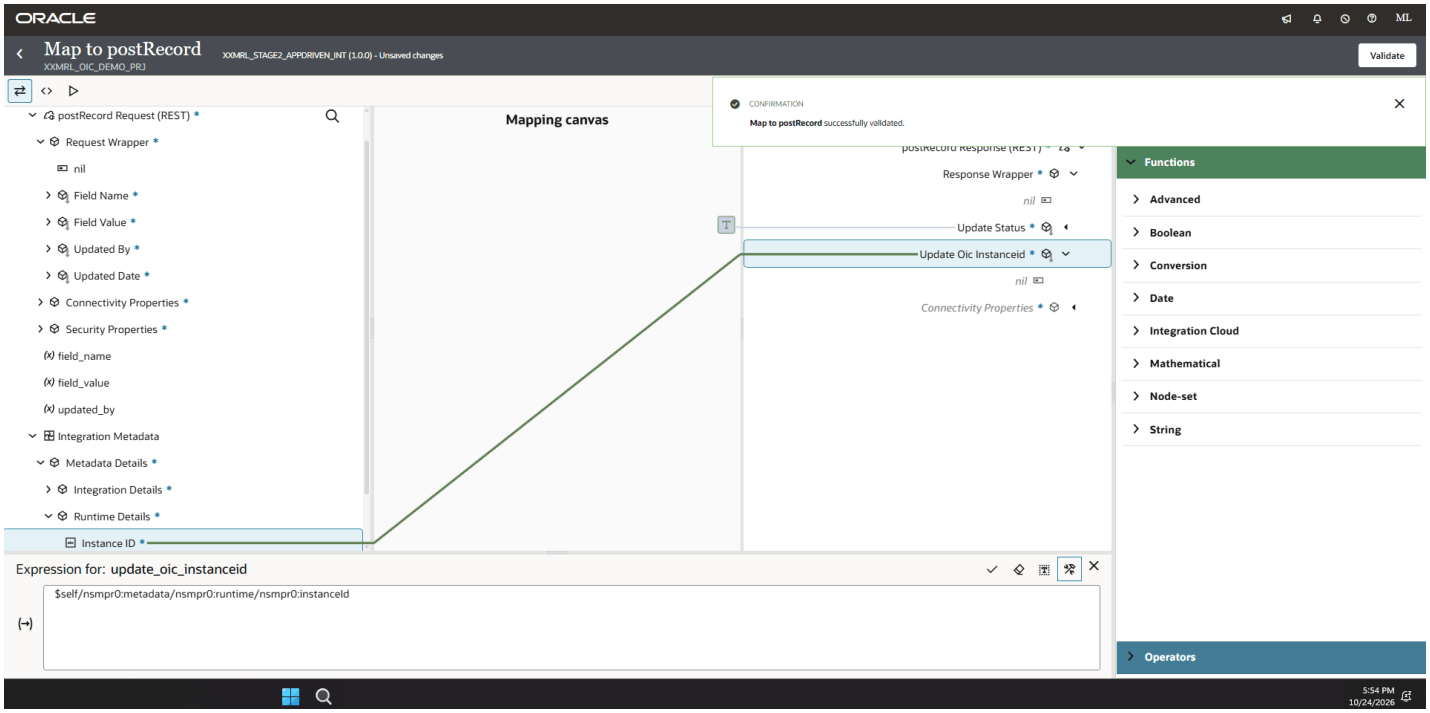
I expanded the target response wrapper, right-clicked each field and chose Create Target Node. I opened the Functions panel on the upper right and clicked Switch to Developer View at the bottom to enter the mapping code:



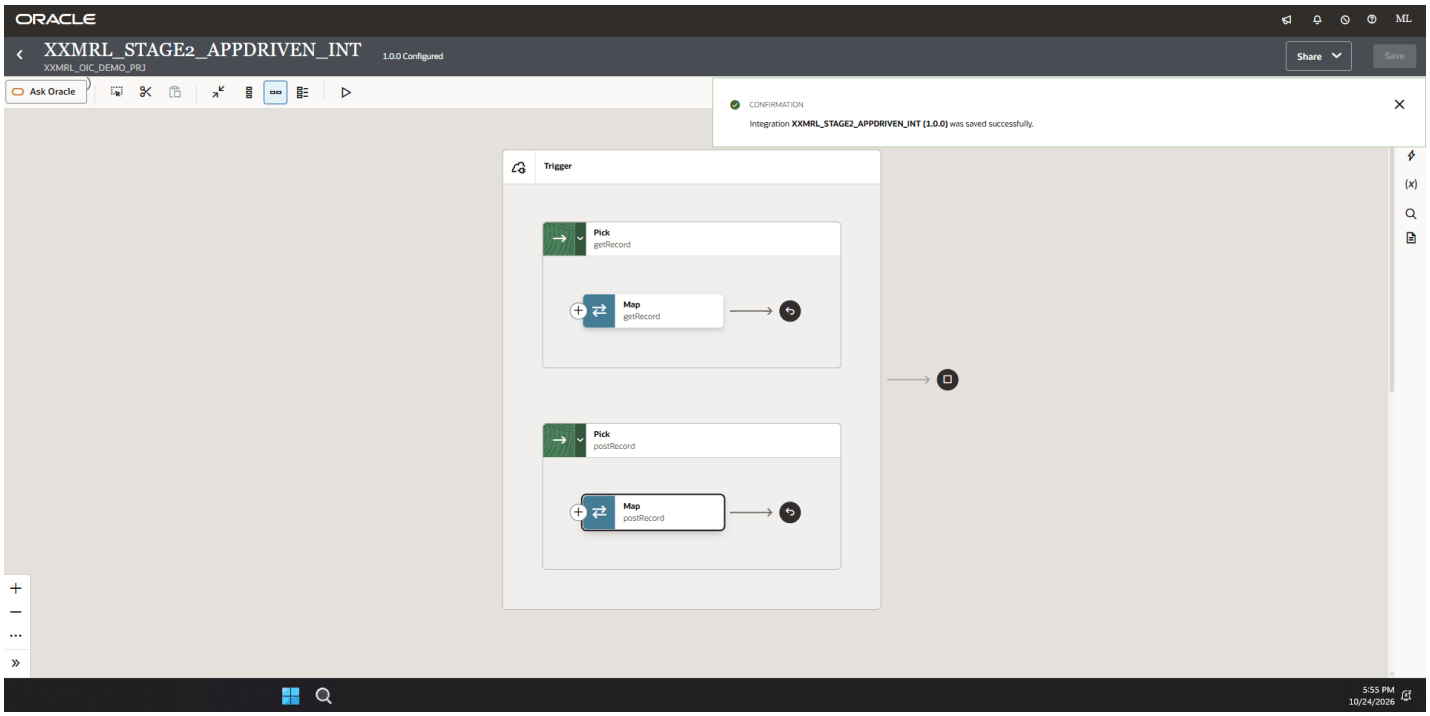
For demo purposes, I set update_status to the hardcoded value ‘SUCCESS’ and update_oic_instanceid to the runtime OIC instance ID from the integration metadata:



I clicked the check mark at the bottom to save, then clicked Validate on the upper right:



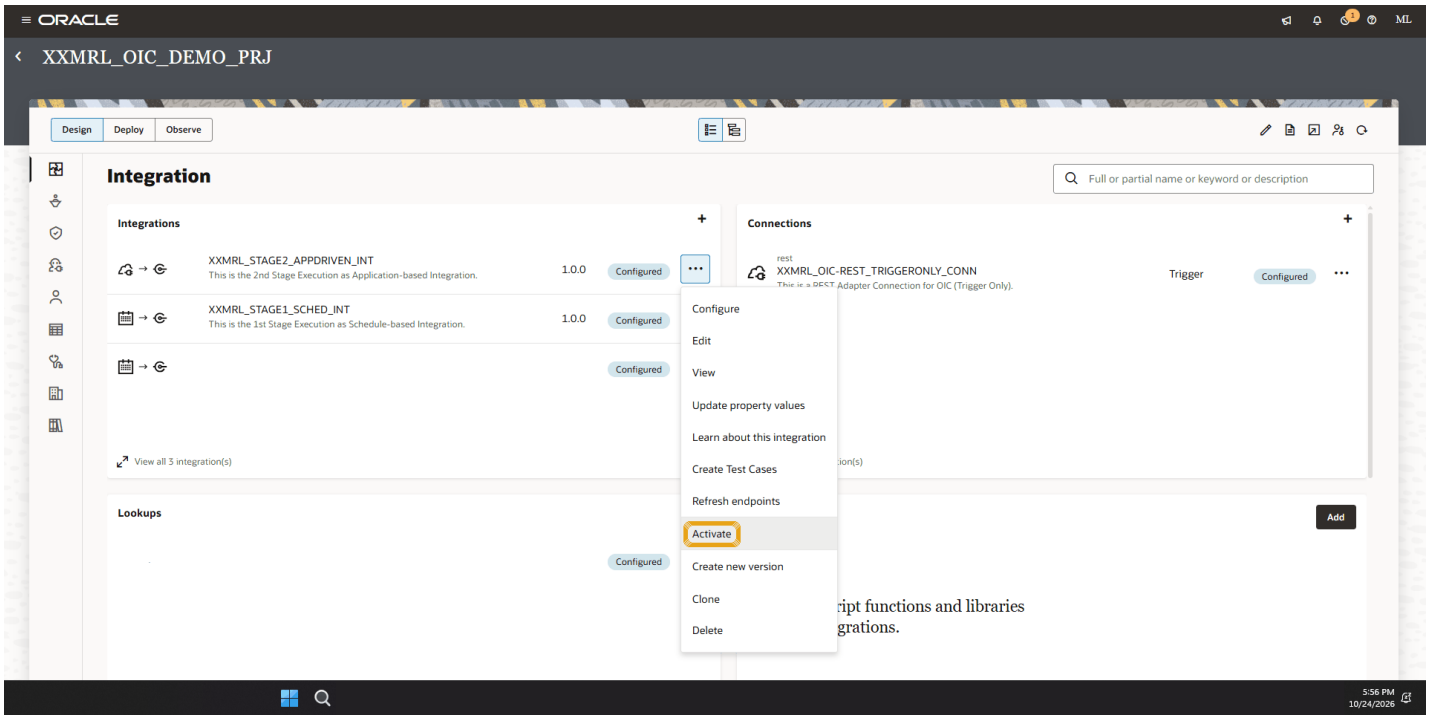
I went back to the main integration and clicked Save:



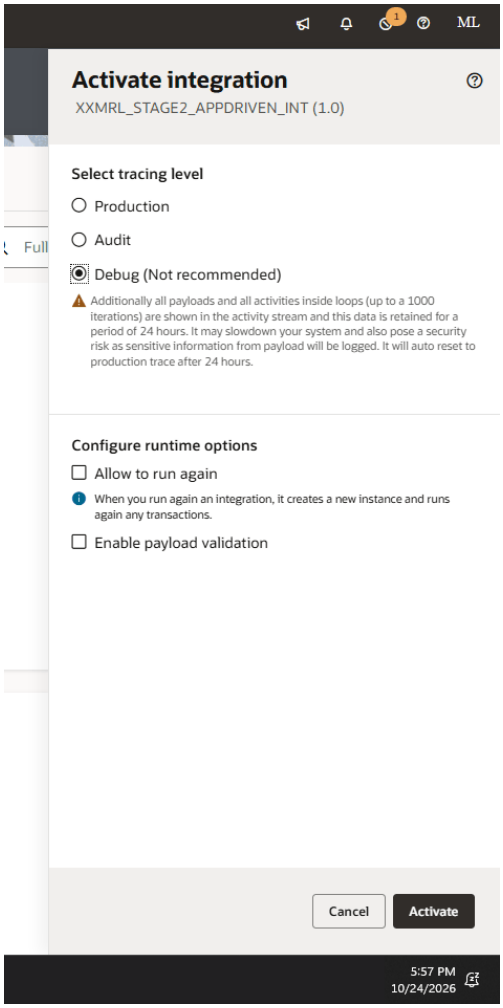
Then I exited the integration.

Part 2.2: Activate and Test the App-Driven Integration

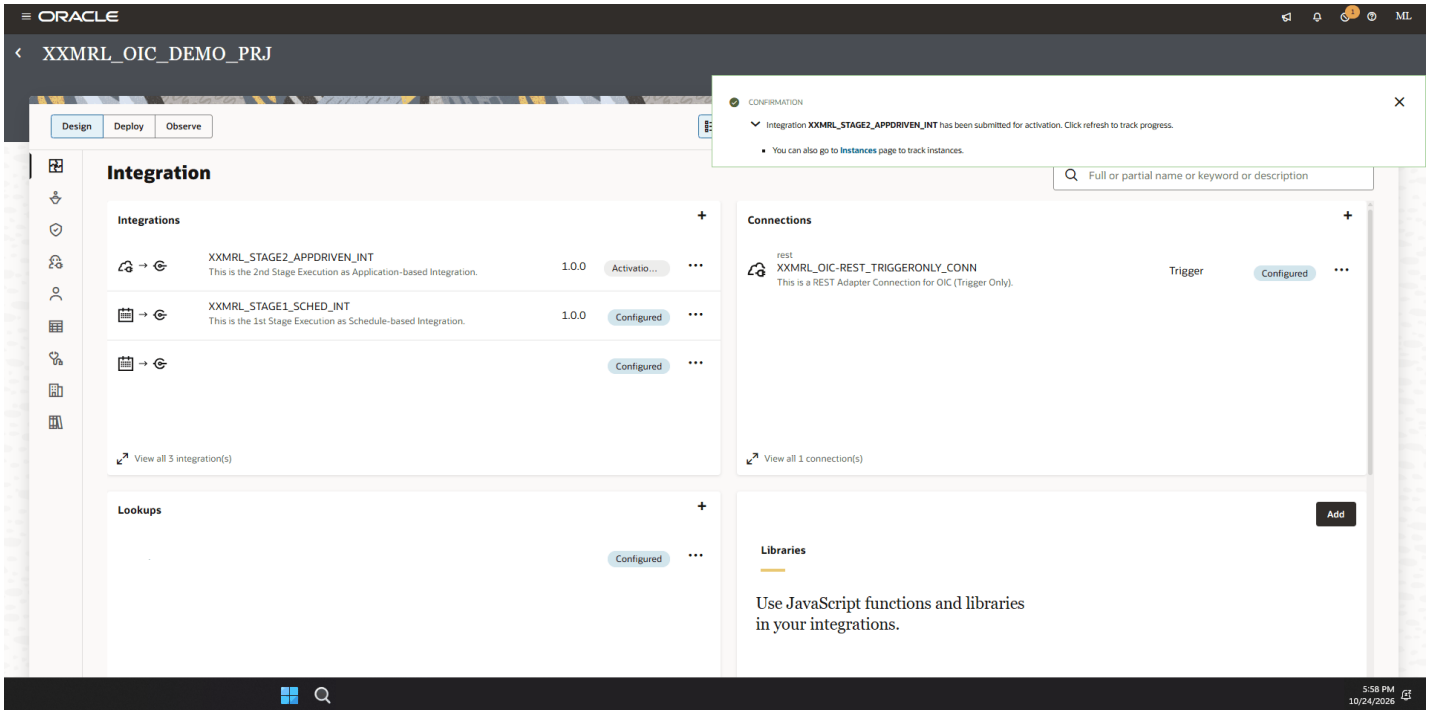
Back in the project, I opened the … menu for the app-driven integration and chose Activate:



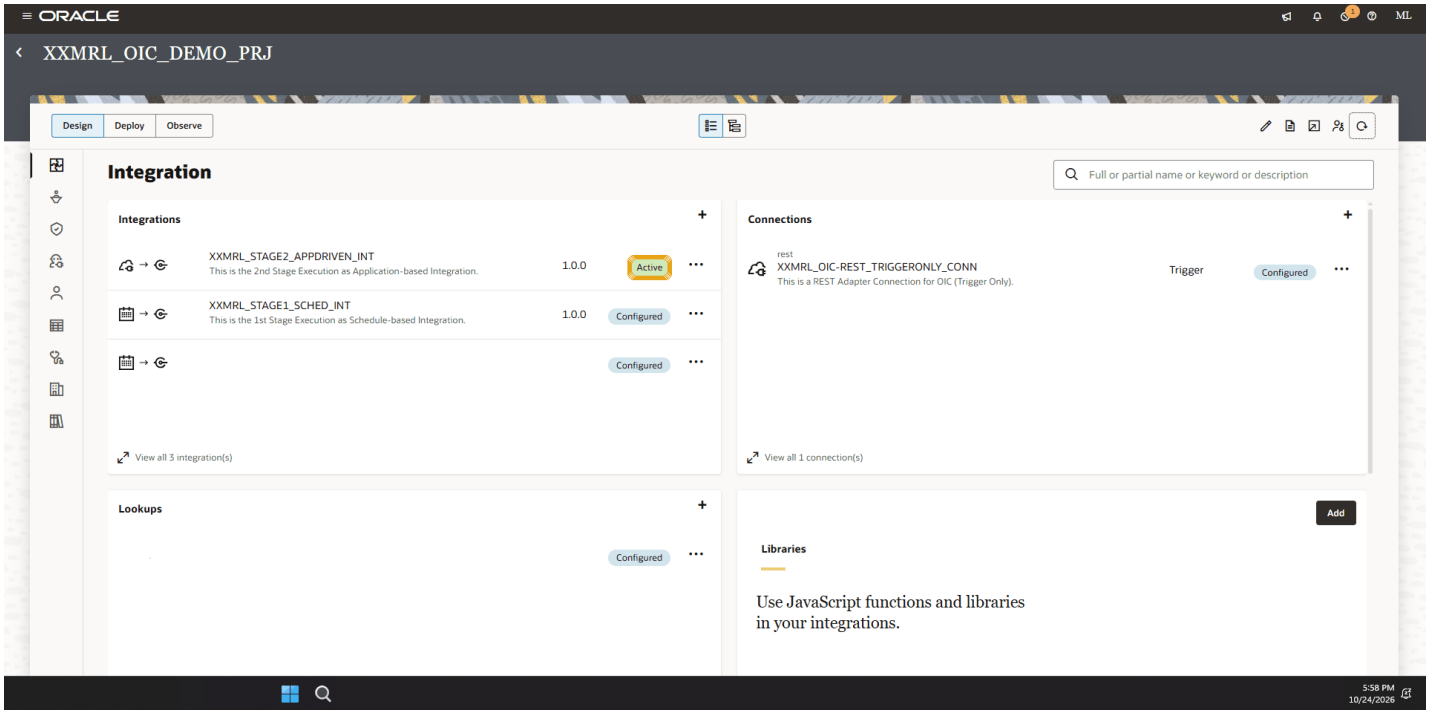
I set the tracing level to Debug, since this was a development demo.



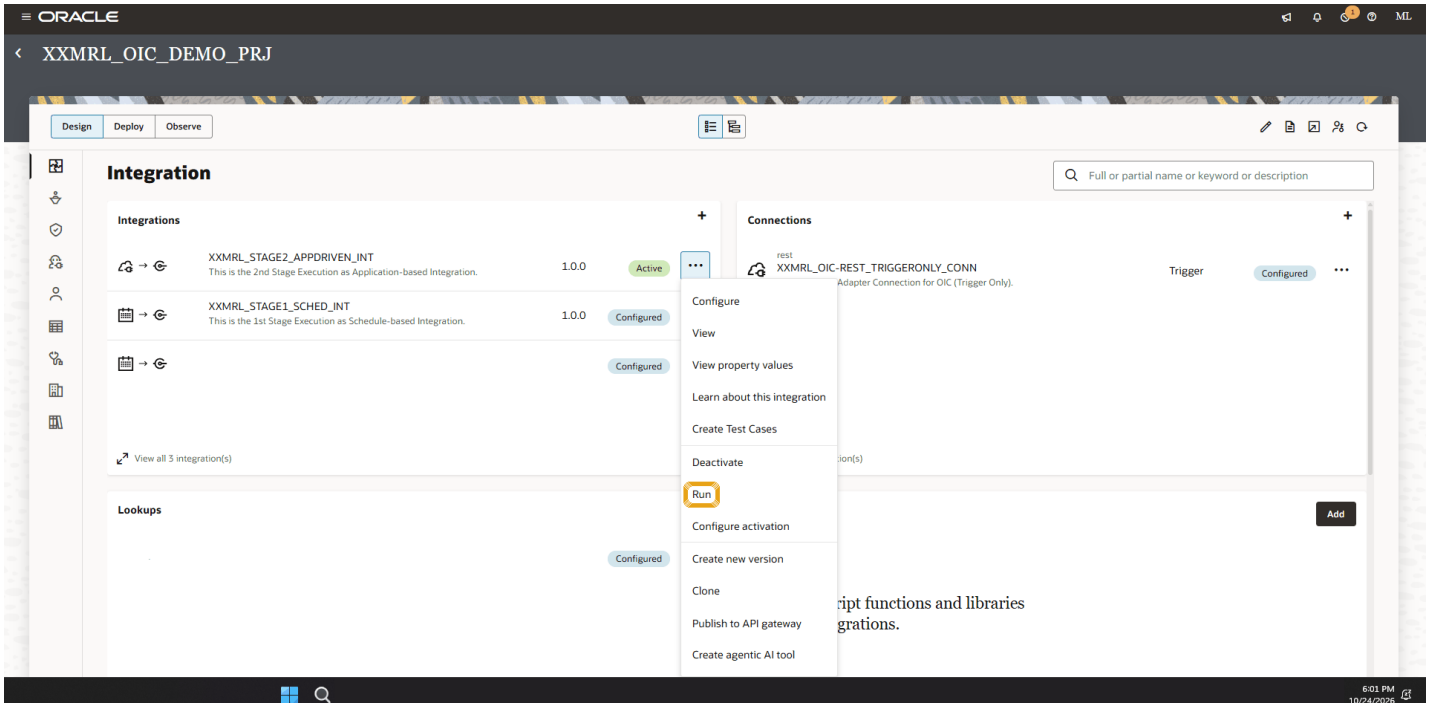
I clicked Activate, and a confirmation message showed that the activation was submitted:



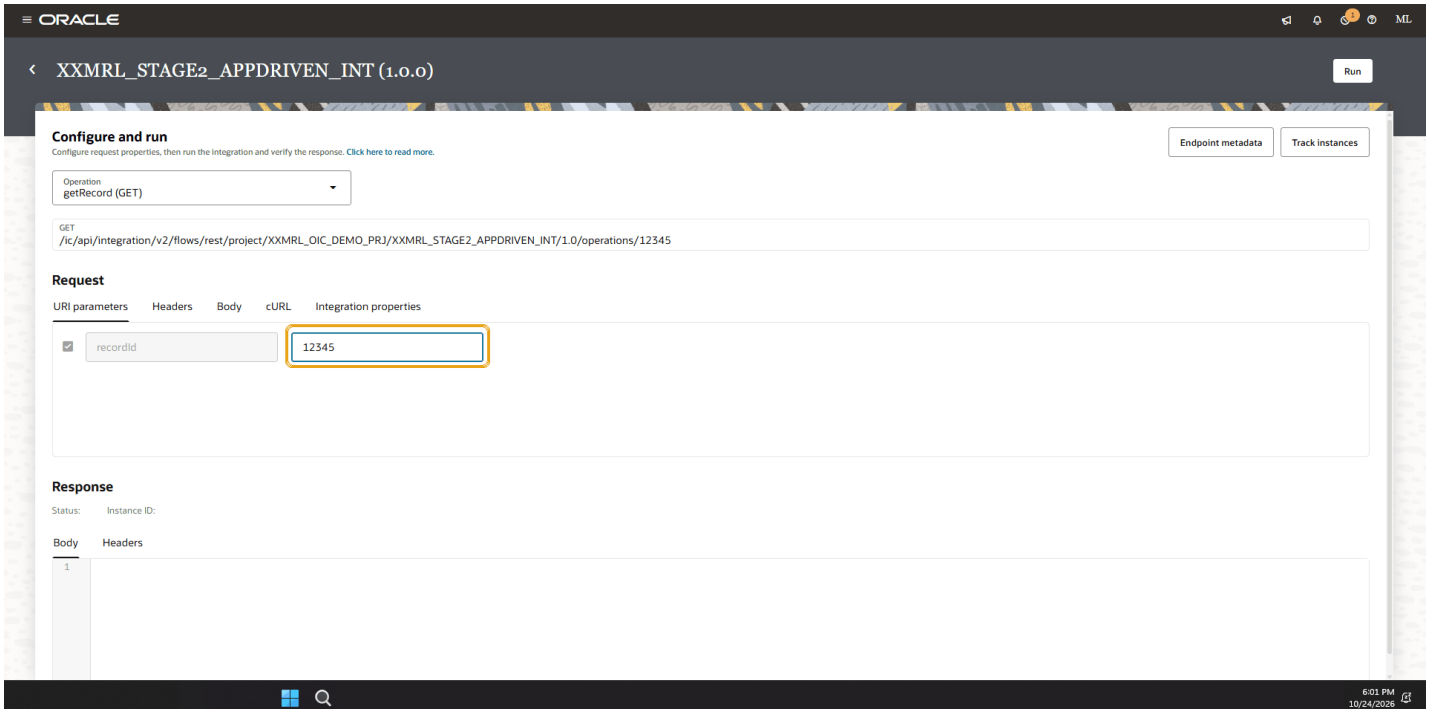
I clicked the refresh button on the upper right to check the status. After a few seconds, the app-driven integration showed as Active:



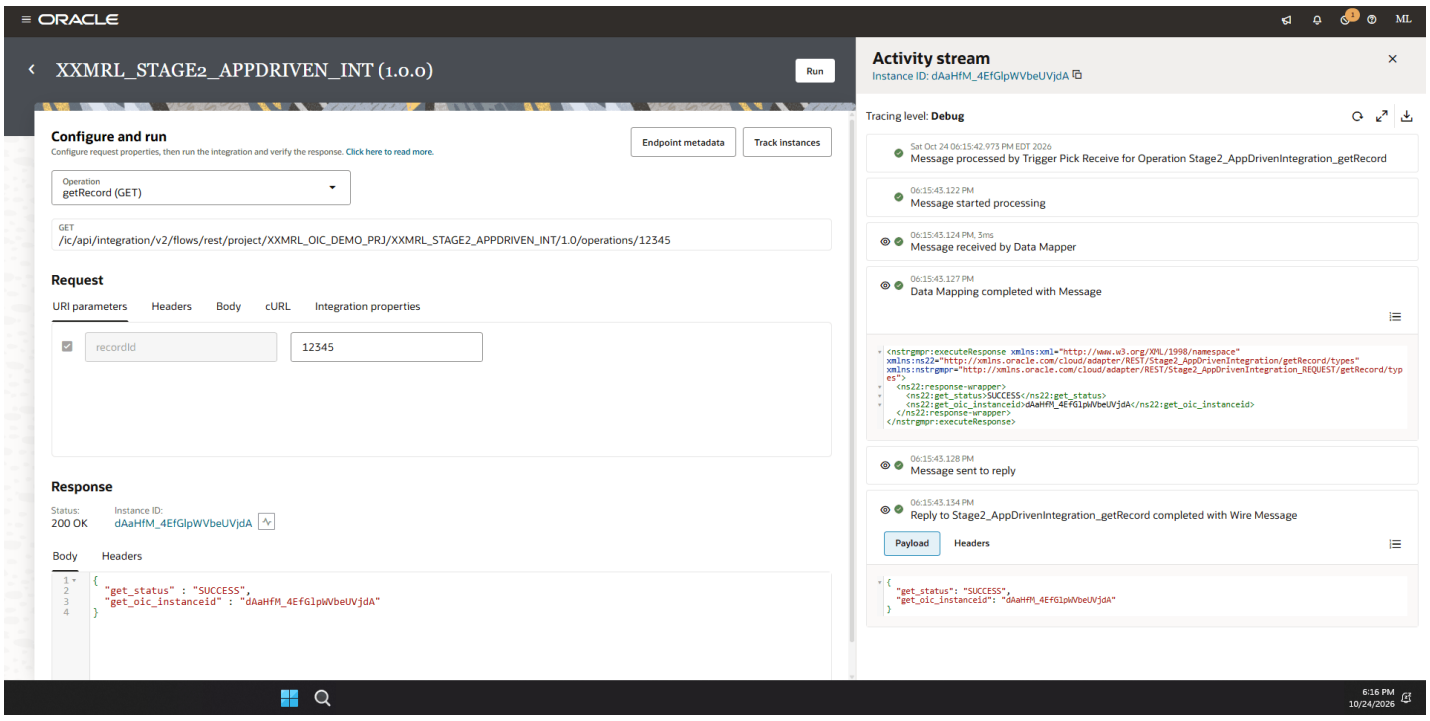
To test the app-driven integration, I opened the … menu and chose Run.



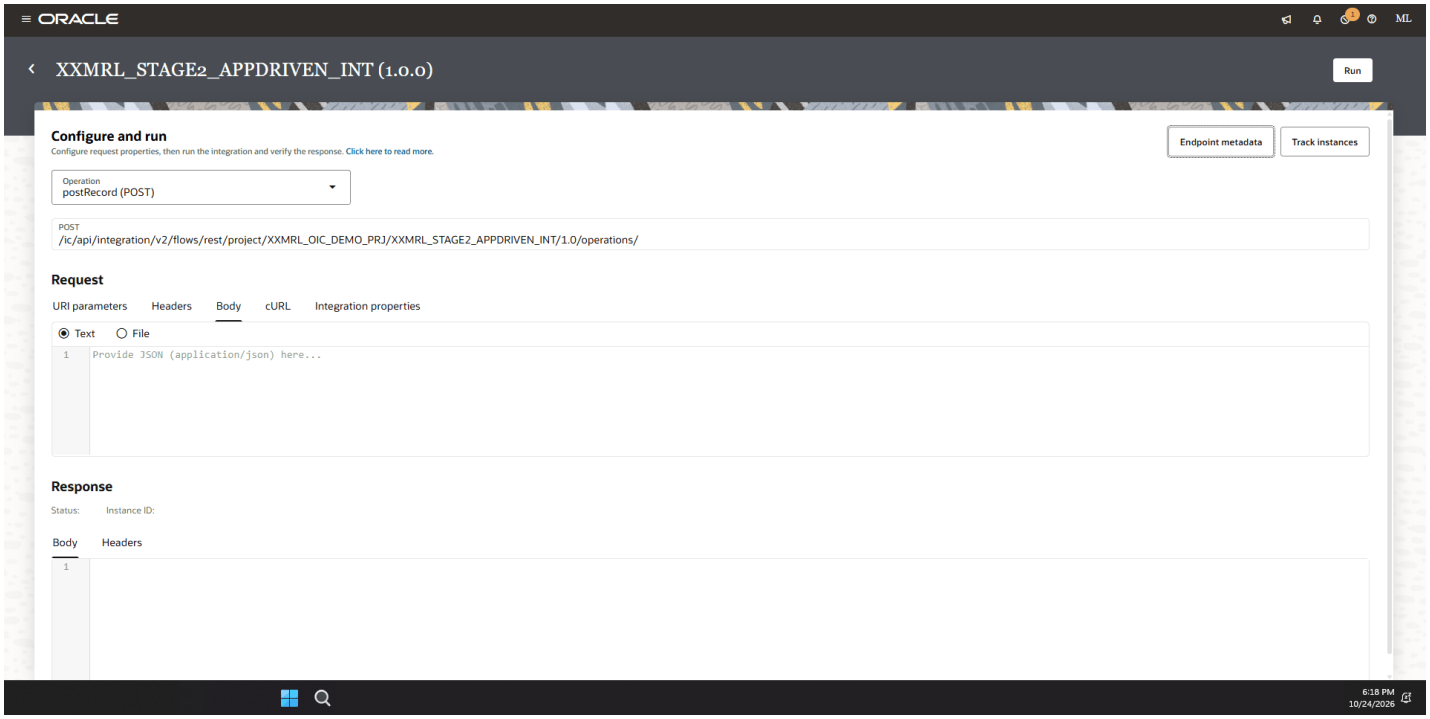
I started with getRecord. In the request form under URI Parameters, I set recordId to 12345 and clicked Run:



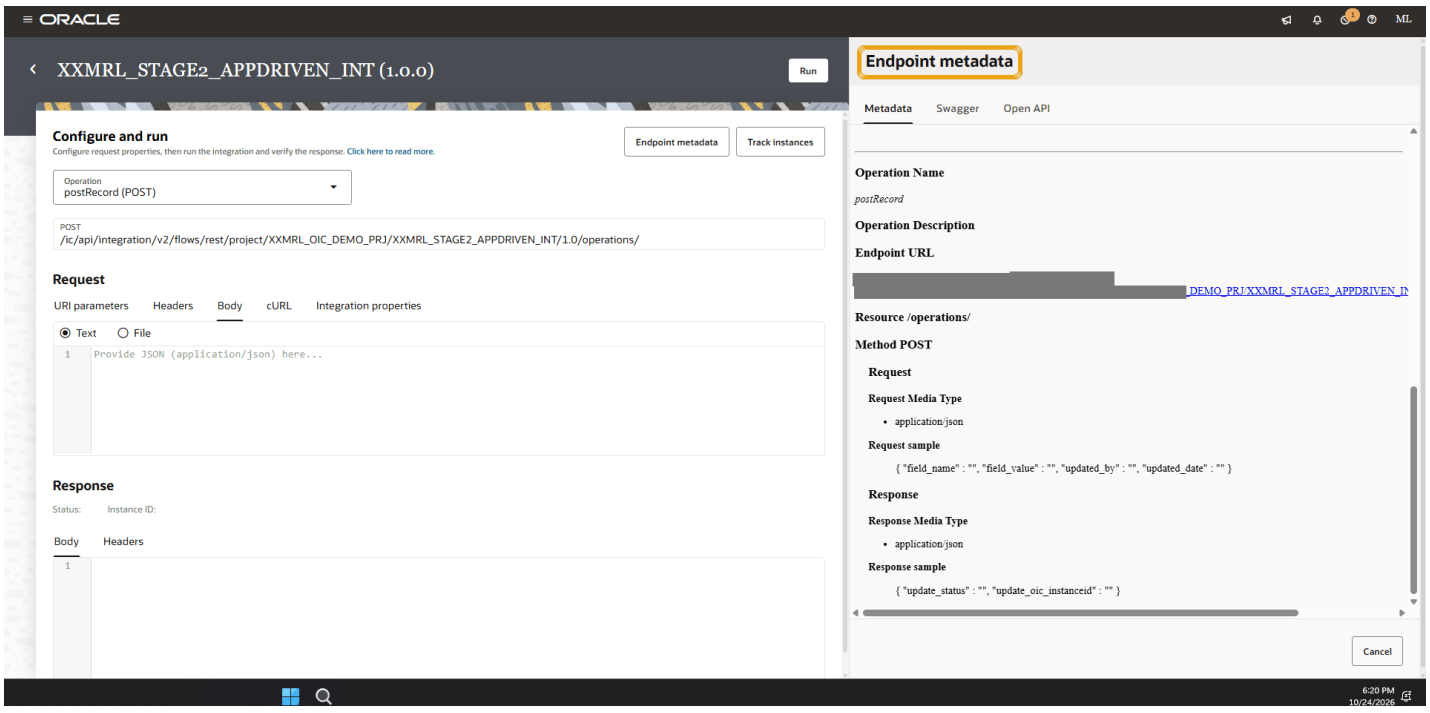
The Response section was populated, and because debug tracing was on, the activity stream on the right showed the detailed audit of the run:



Next, I tested the postRecord operation:

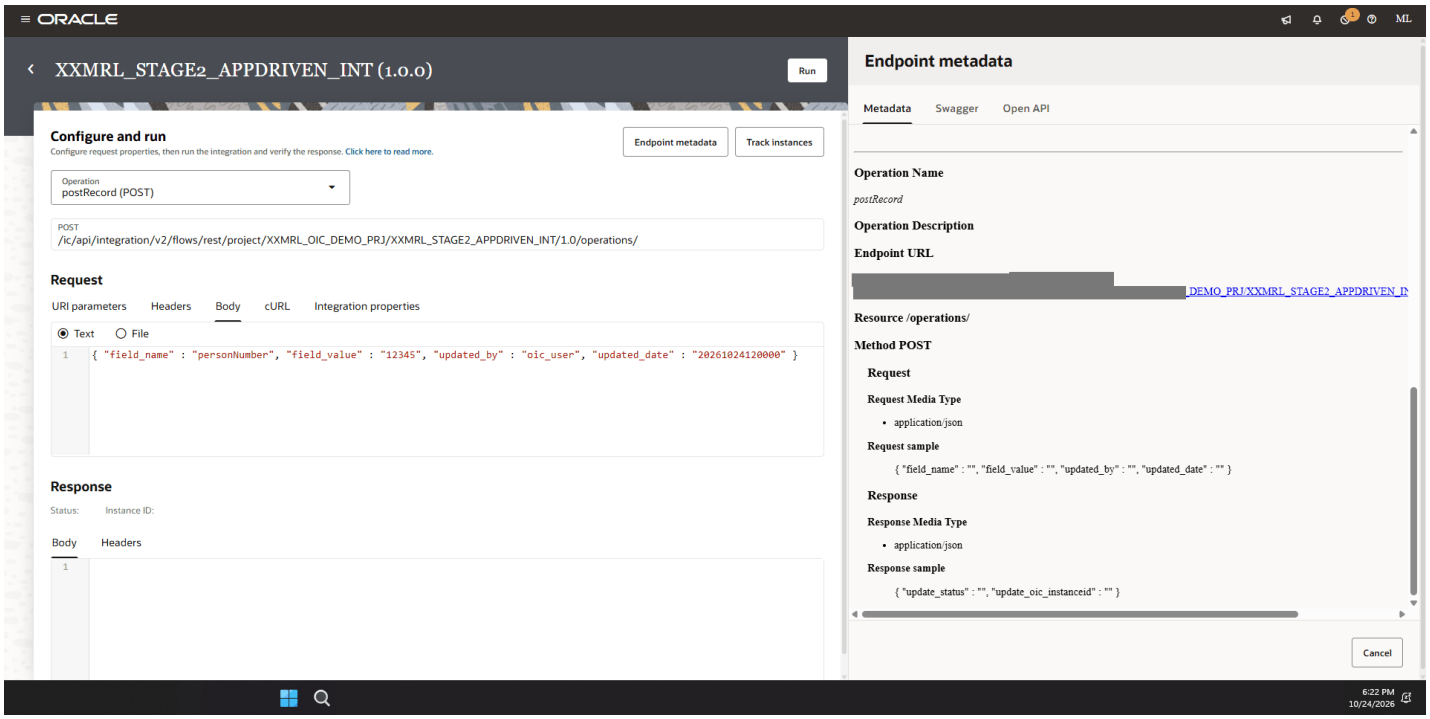


Because this is a POST, the request needed a body. The body editor started empty, so I opened Endpoint Metadata to see the expected request body structure:

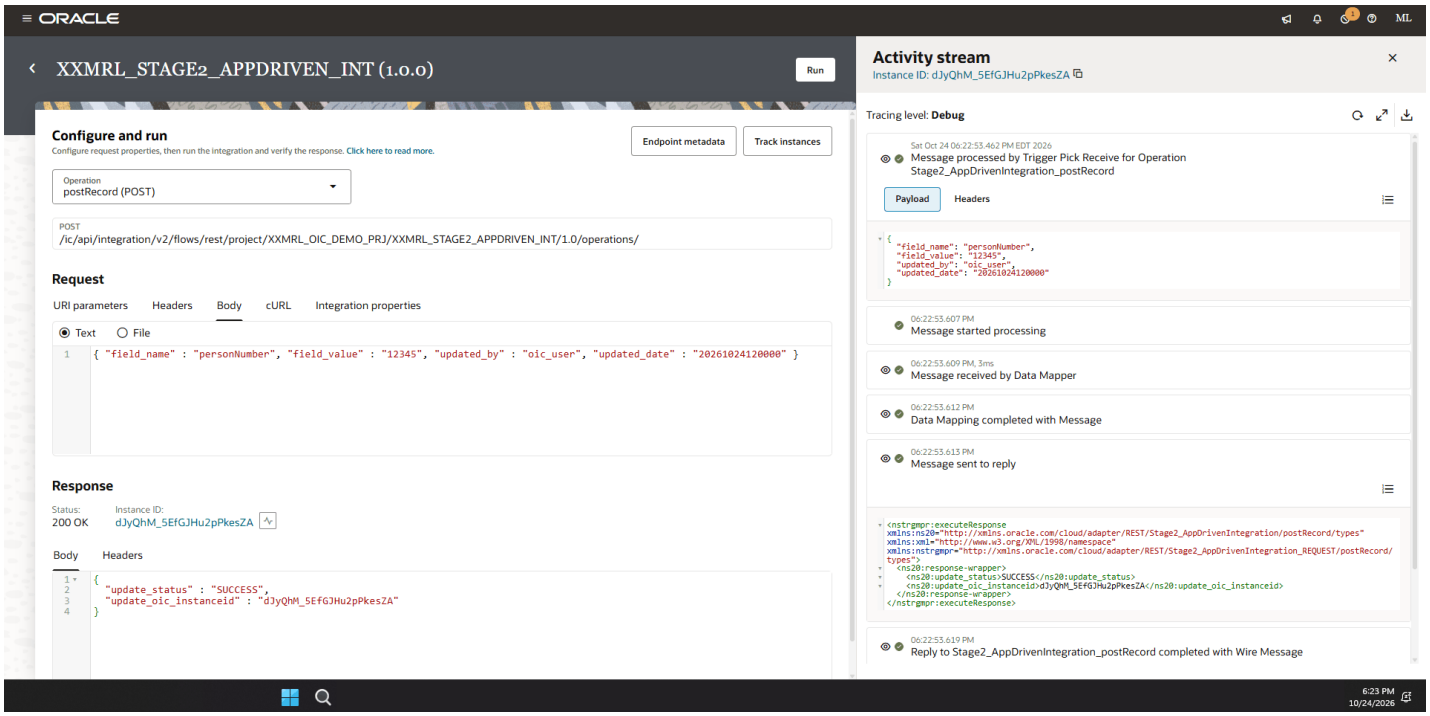


For demo purposes, I used this request body:

```
{ "field_name" : "personNumber", "field_value" : "12345", "updated_by" : "oic_user", "updated_date" :
"20261024120000" }
```



I clicked Run. The Response section showed the response payload I defined during the build, and the activity stream showed how the request was received and processed, all the way to the response values:

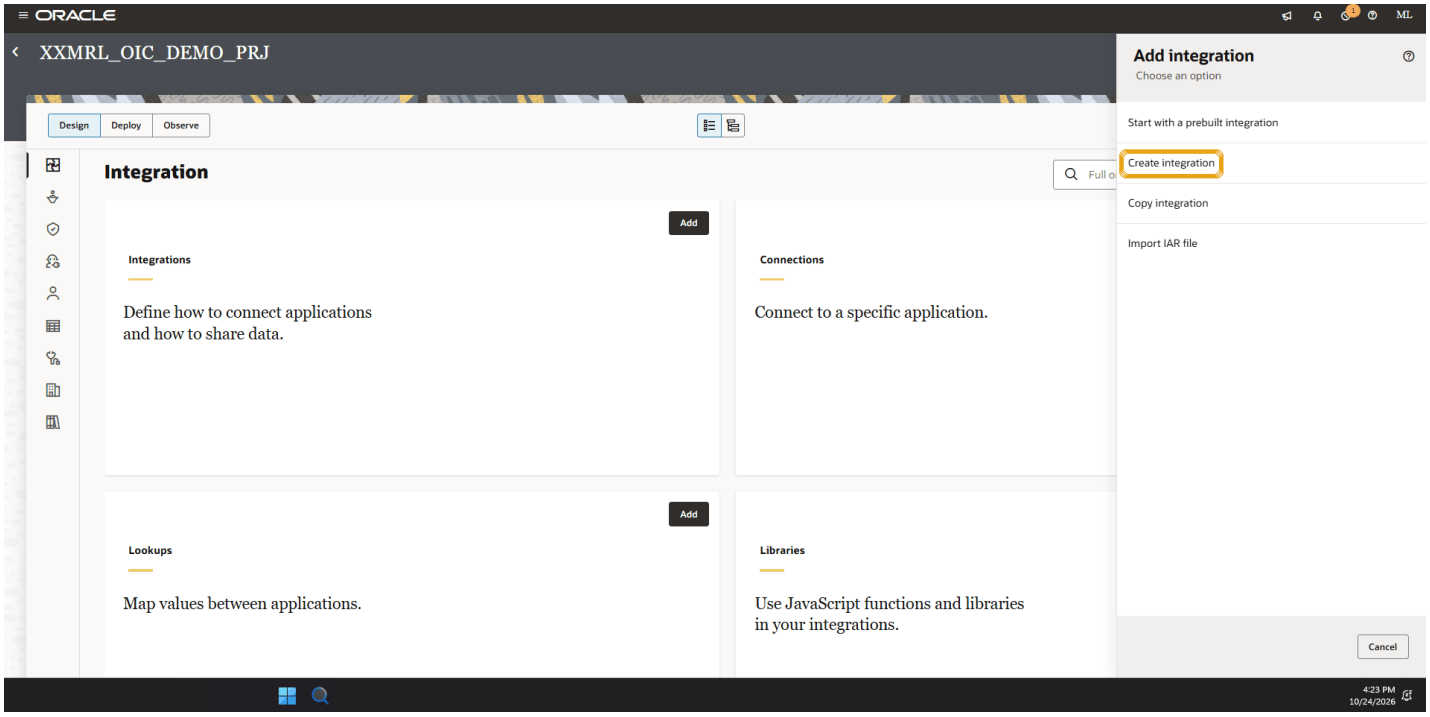


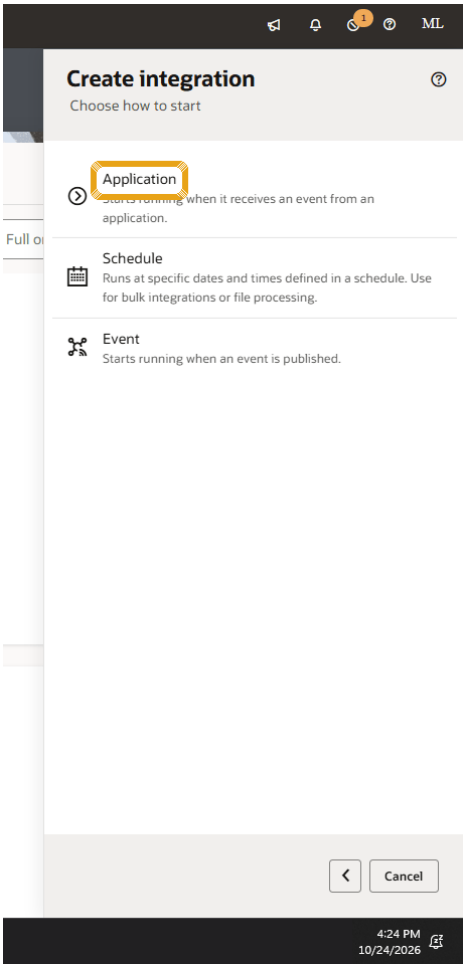
Part 3.1: Create the Schedule Integration

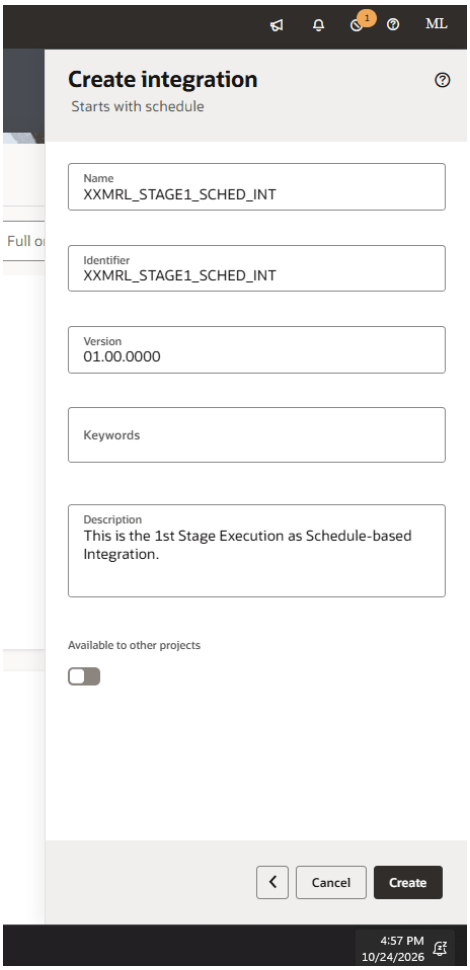
In the project, I went to Integrations, clicked Add, chose Create integration and selected Schedule:

Name: XXMRL_STAGE1_SCHED_INT

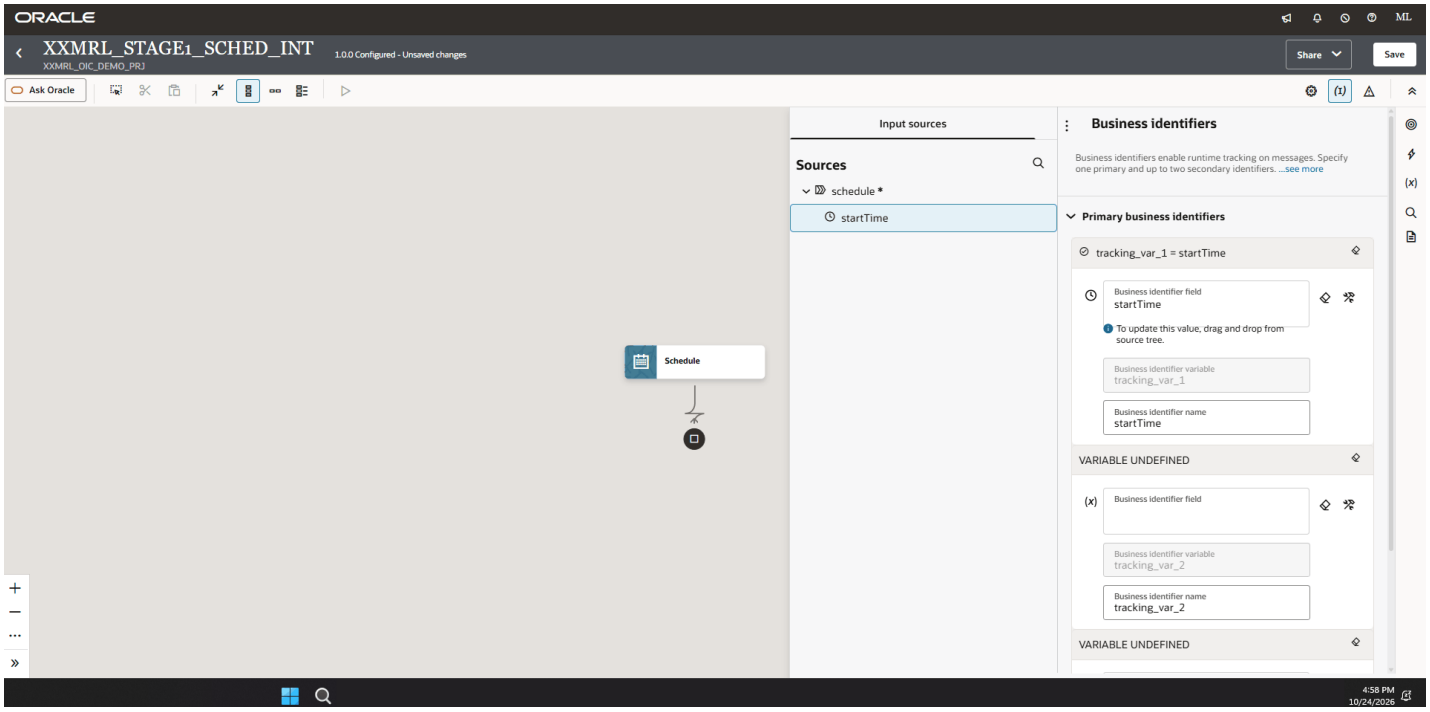
Description: This is the 1st Stage Execution as Schedule-based Integration.



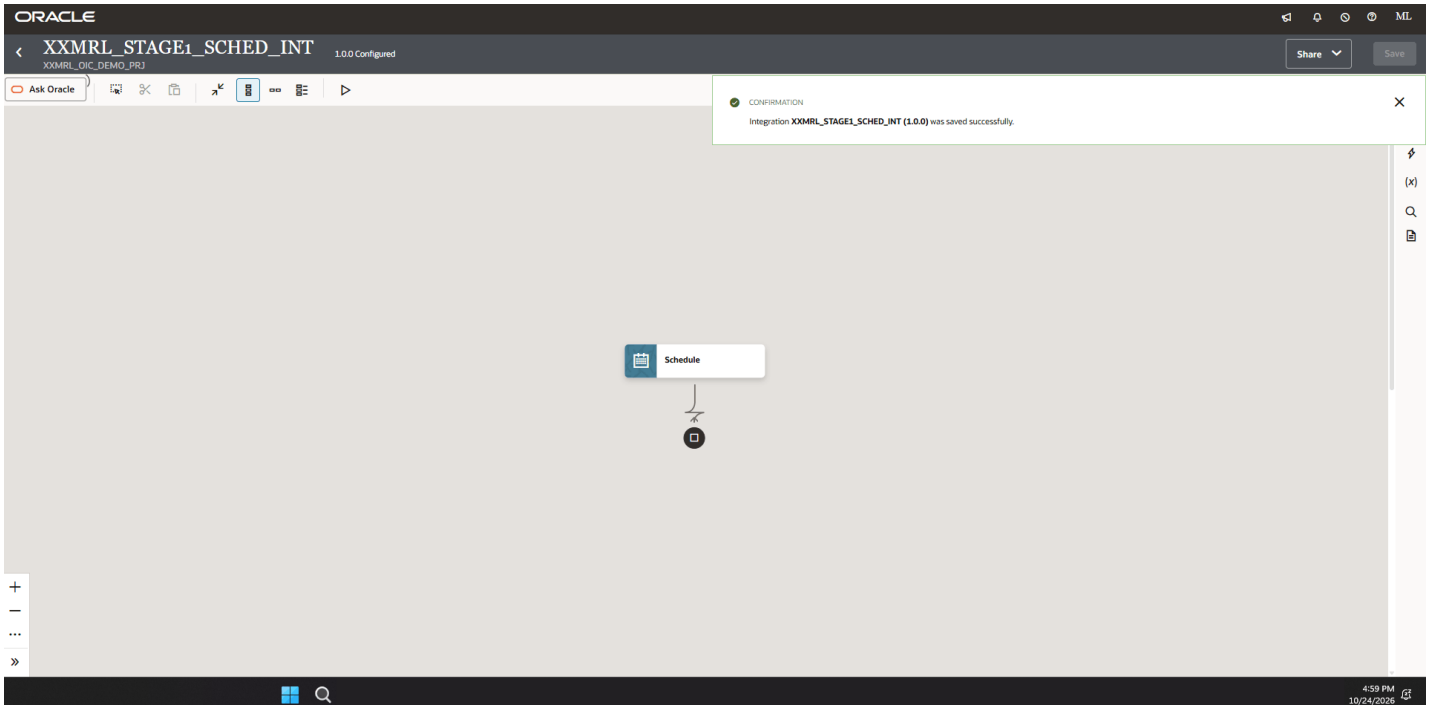




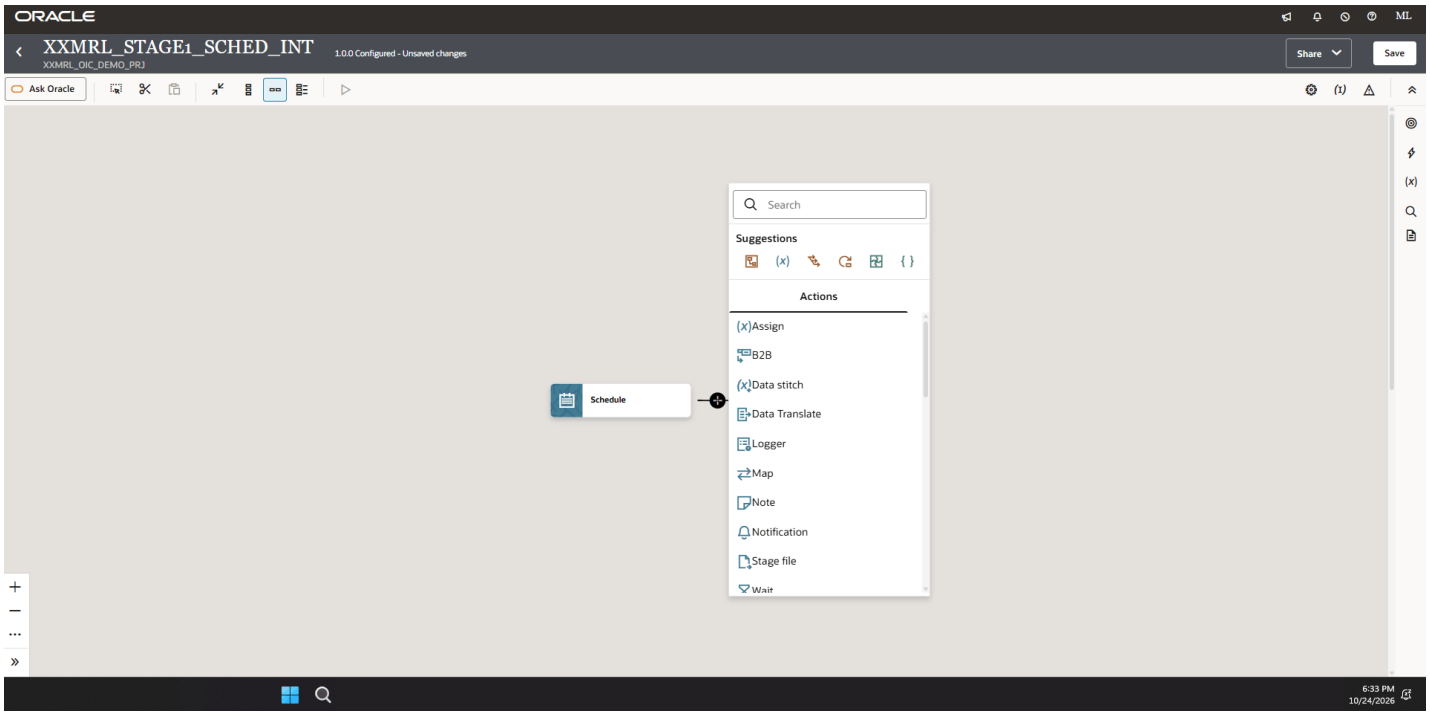
I assigned the business identifier:



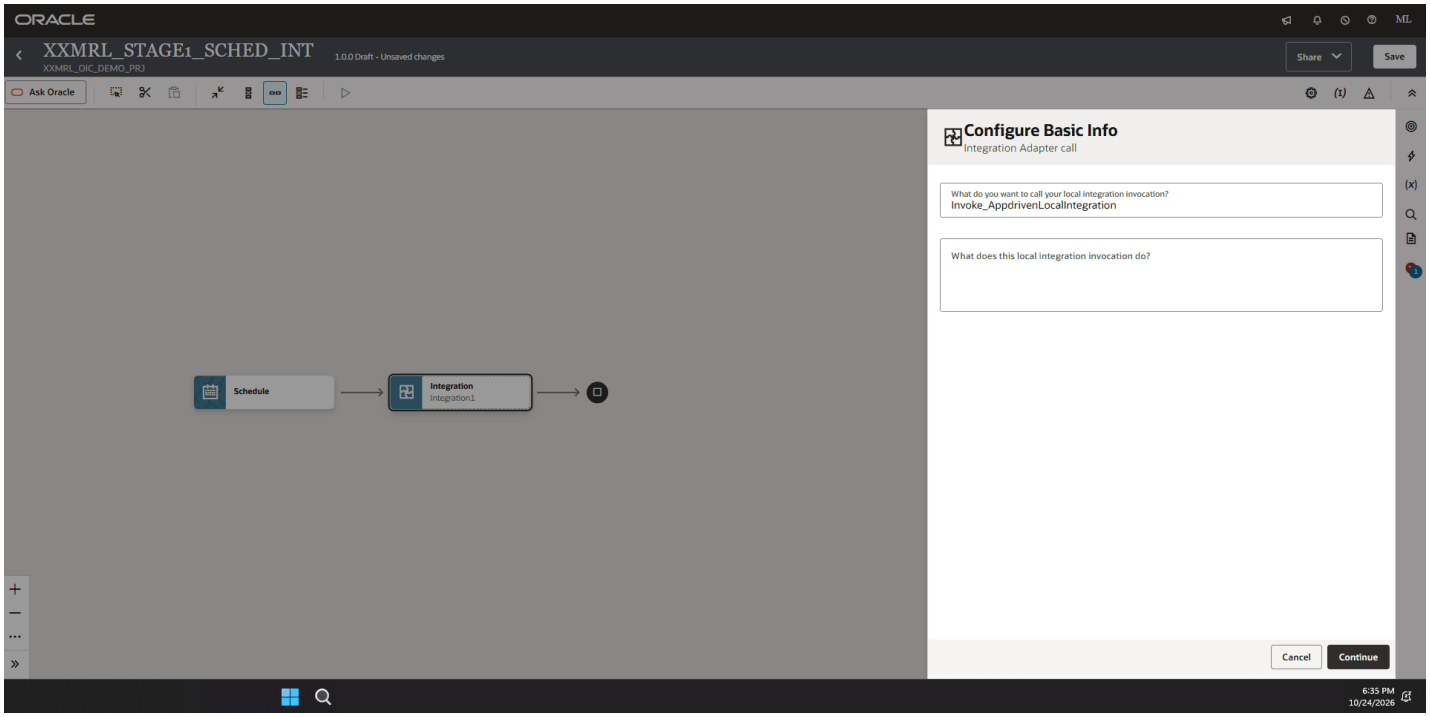
I saved after every change so I wouldn’t lose my work:

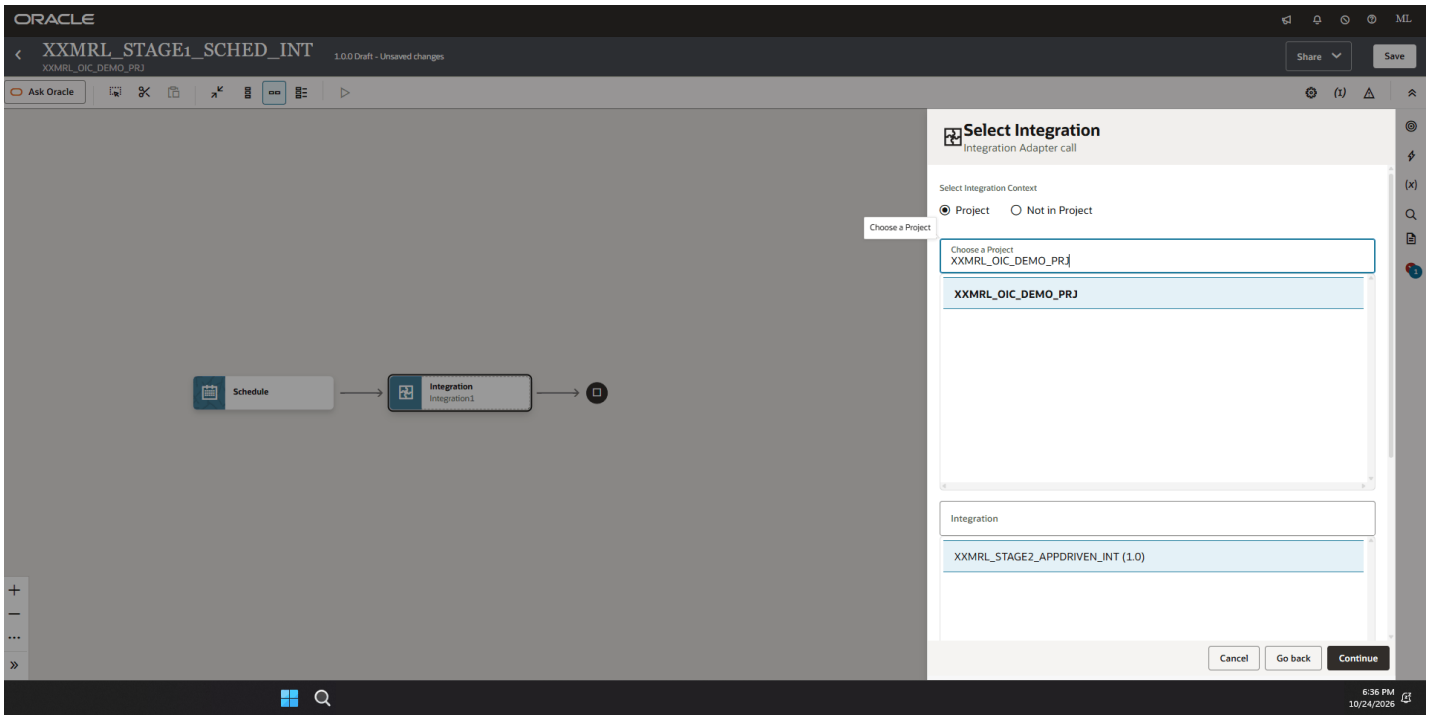


I hovered next to the Schedule action and clicked the Integration icon under Suggestions:

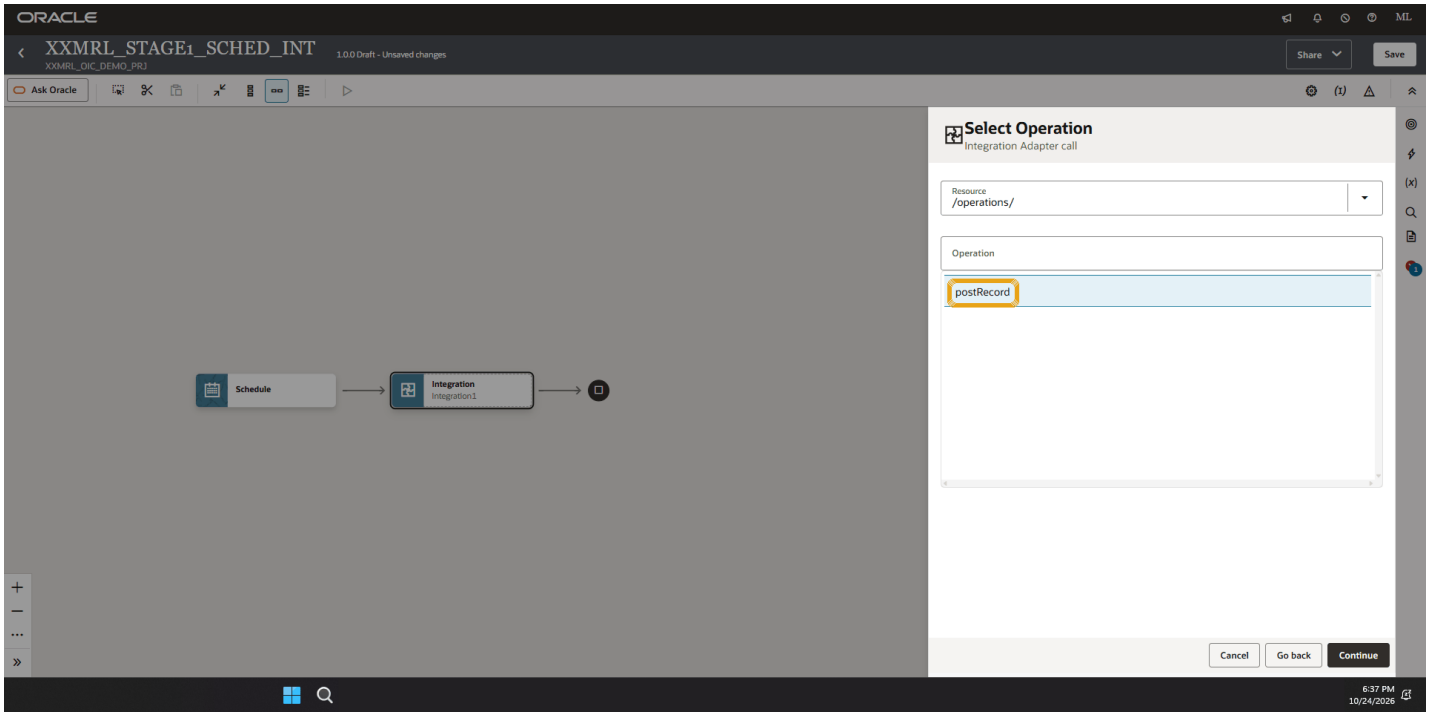


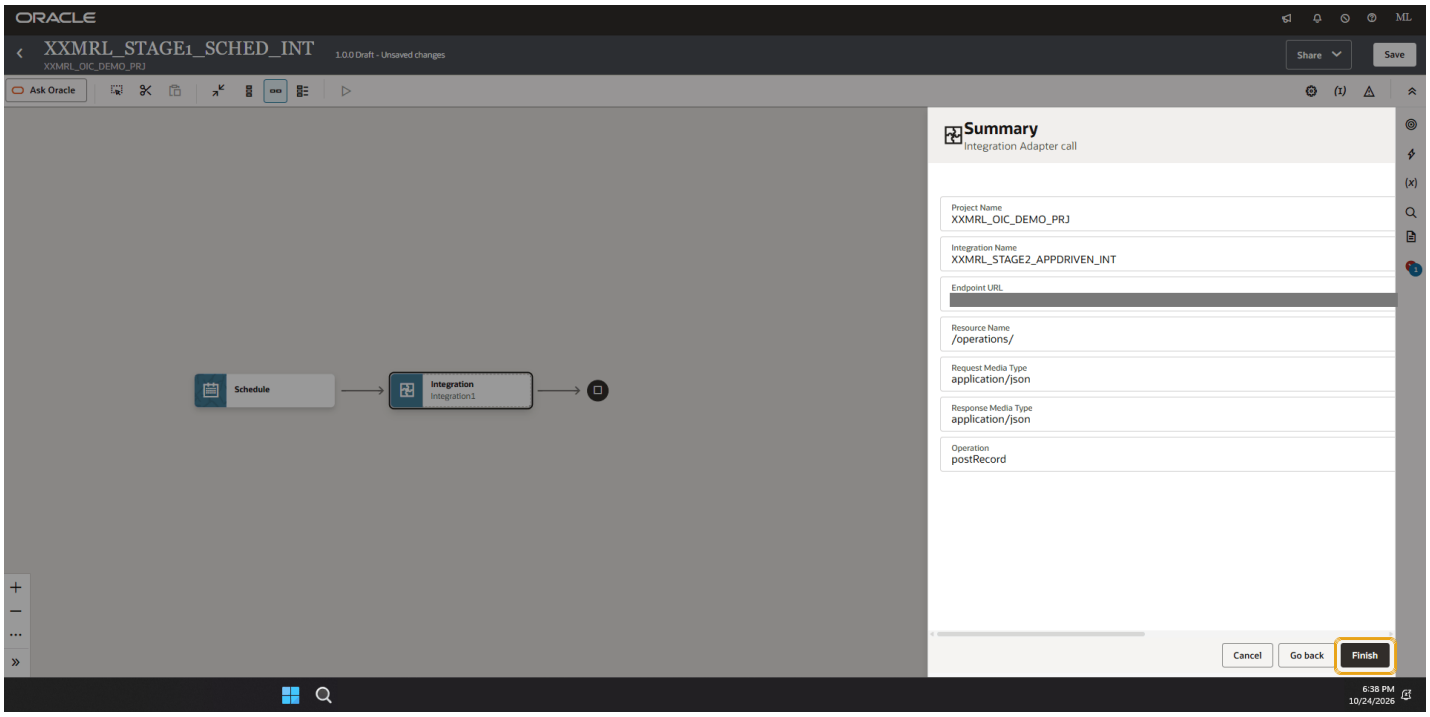
I configured the Local Integration adapter by naming the invoke (Invoke_AppdrivenLocalIntegration) and selecting the project and integration to call:



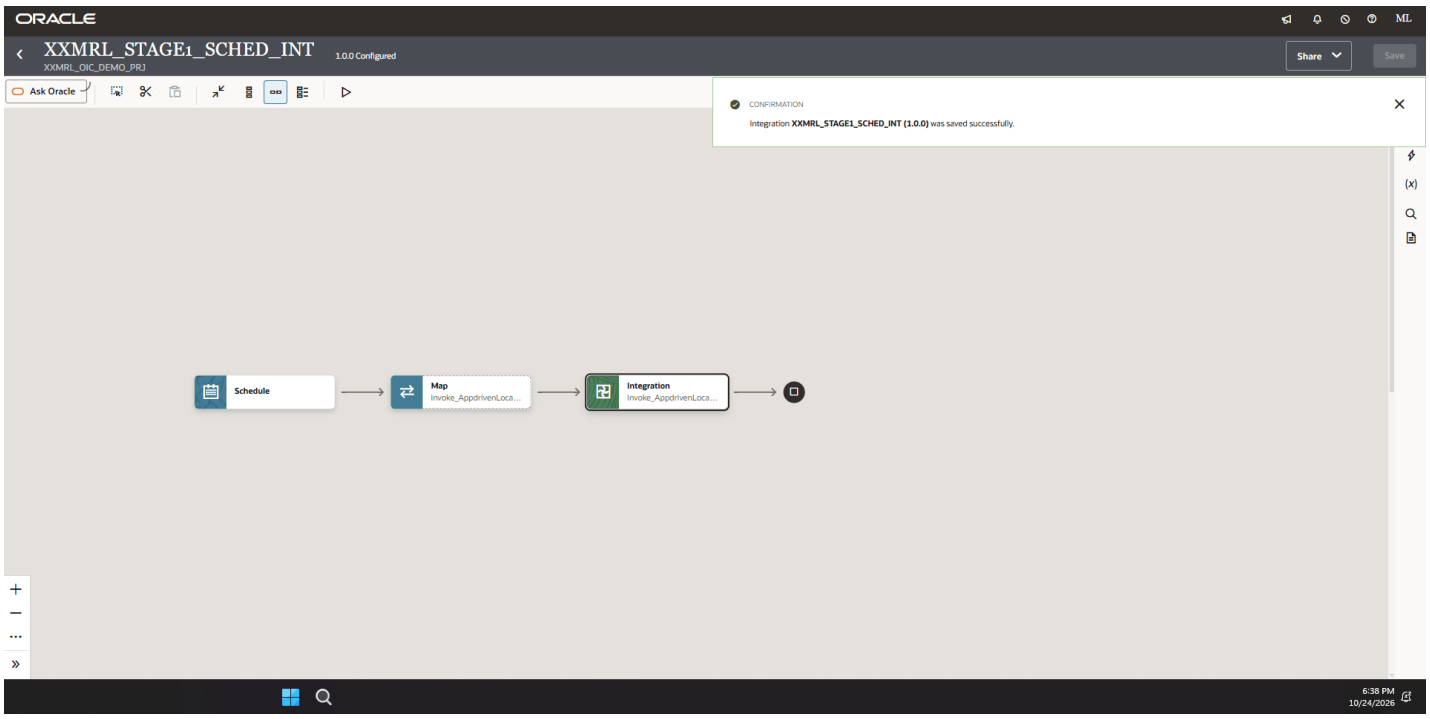


For demo purposes, I had the schedule integration call the postRecord (POST) operation of the app-driven integration:

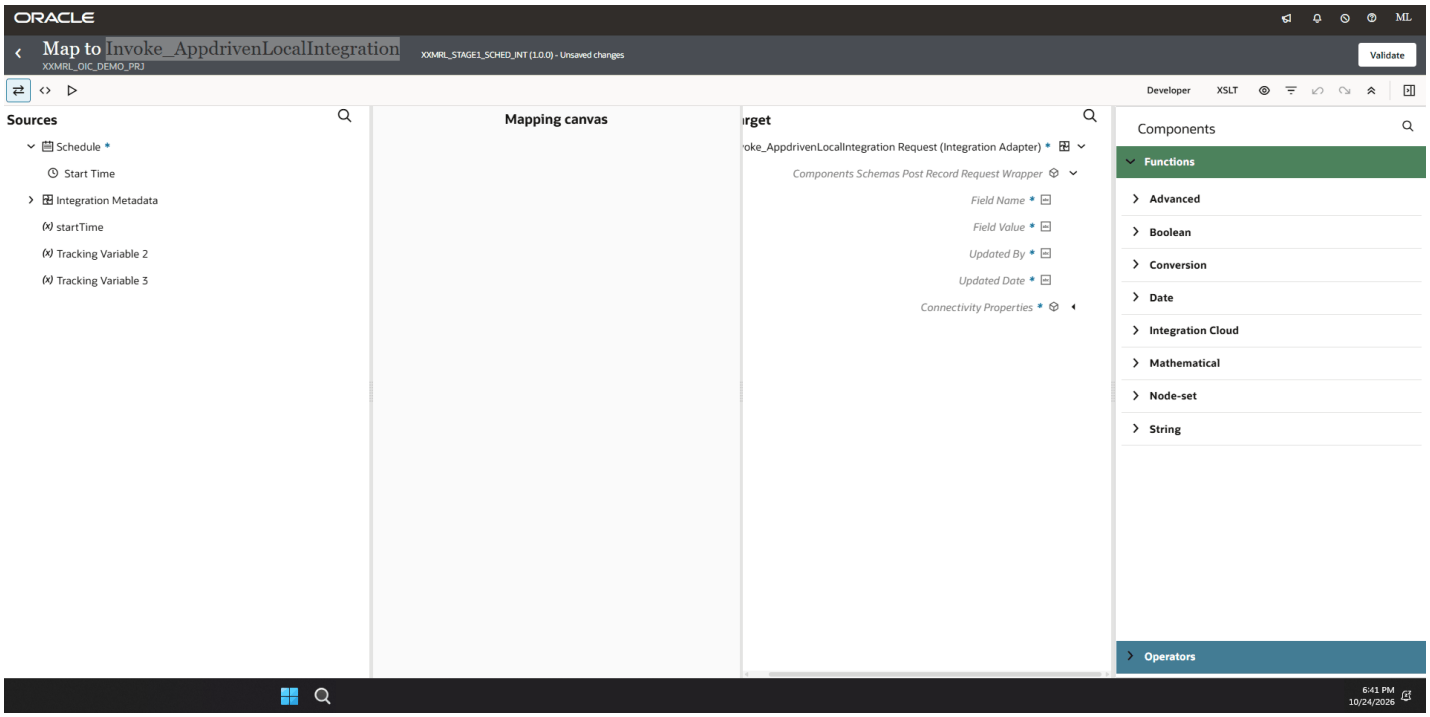




I reviewed the summary and clicked Finish. Back on the orchestration page, I clicked Save:



I opened the mapper of the Invoke_AppdrivenLocalIntegration action, expanded the request wrapper under Target, and opened the Functions panel:



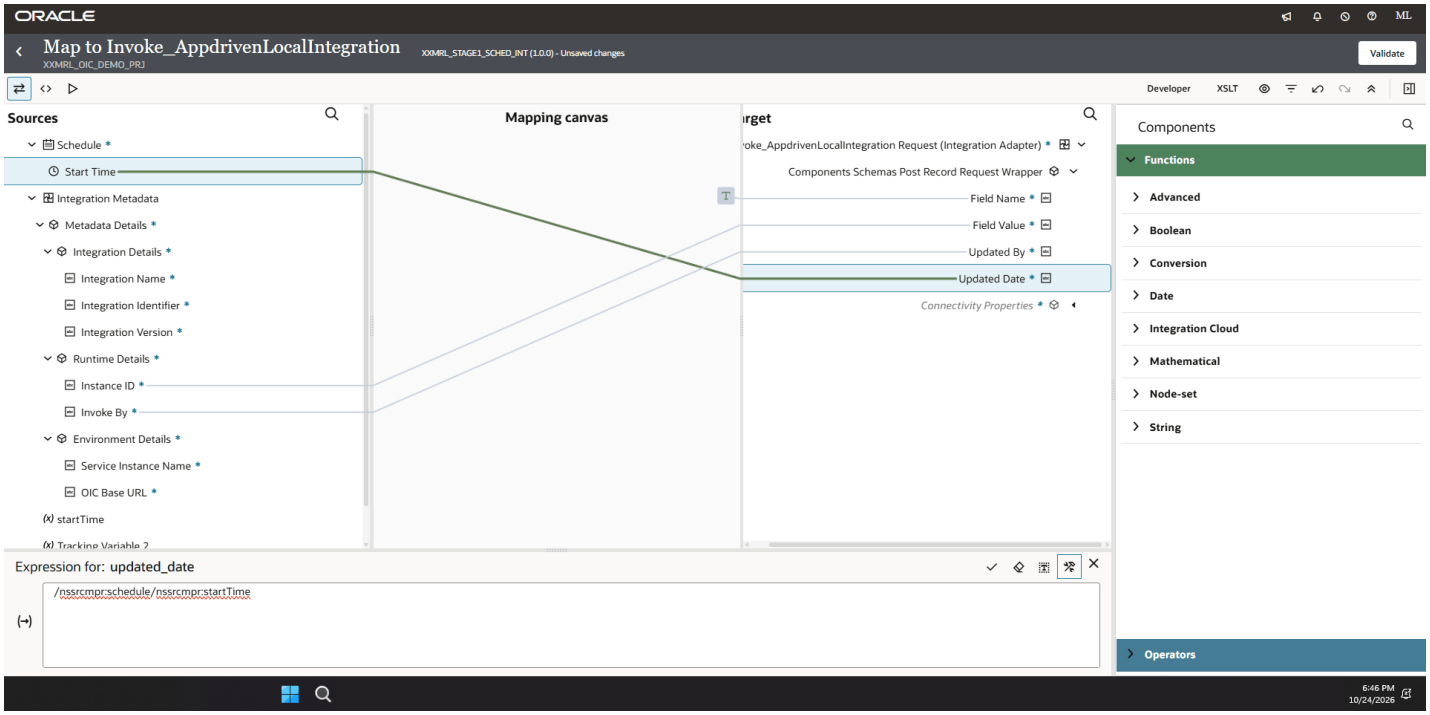
I right-clicked each request wrapper field and chose Create Target Node, then clicked the tool icon at the bottom to switch to Developer View and entered the mappings. For demo purposes, I set:

field_name = the hardcoded value ‘SCHED_OIC_INSTANCEID’

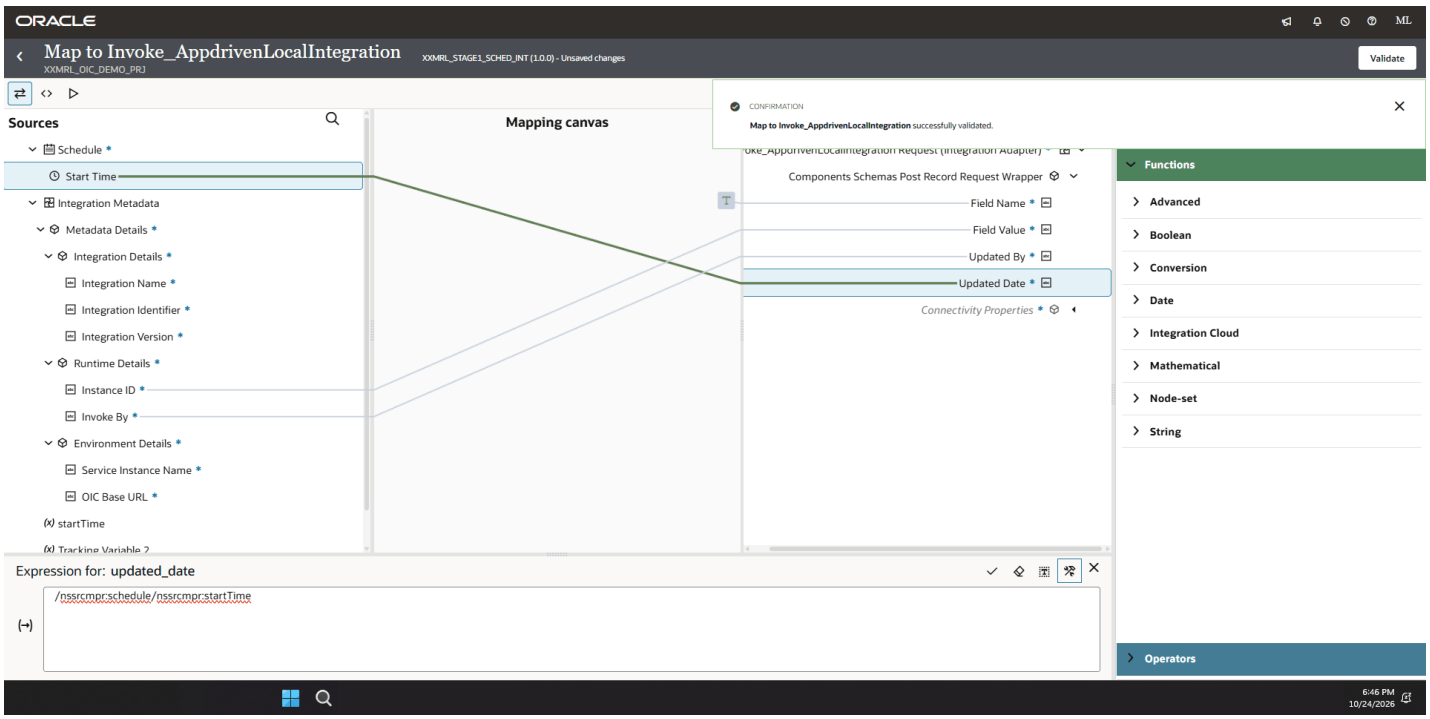
field_value = the schedule integration’s OIC instance ID at runtime

updated_by = the invokedBy runtime metadata value

updated_date = the schedule’s startTime value at runtime



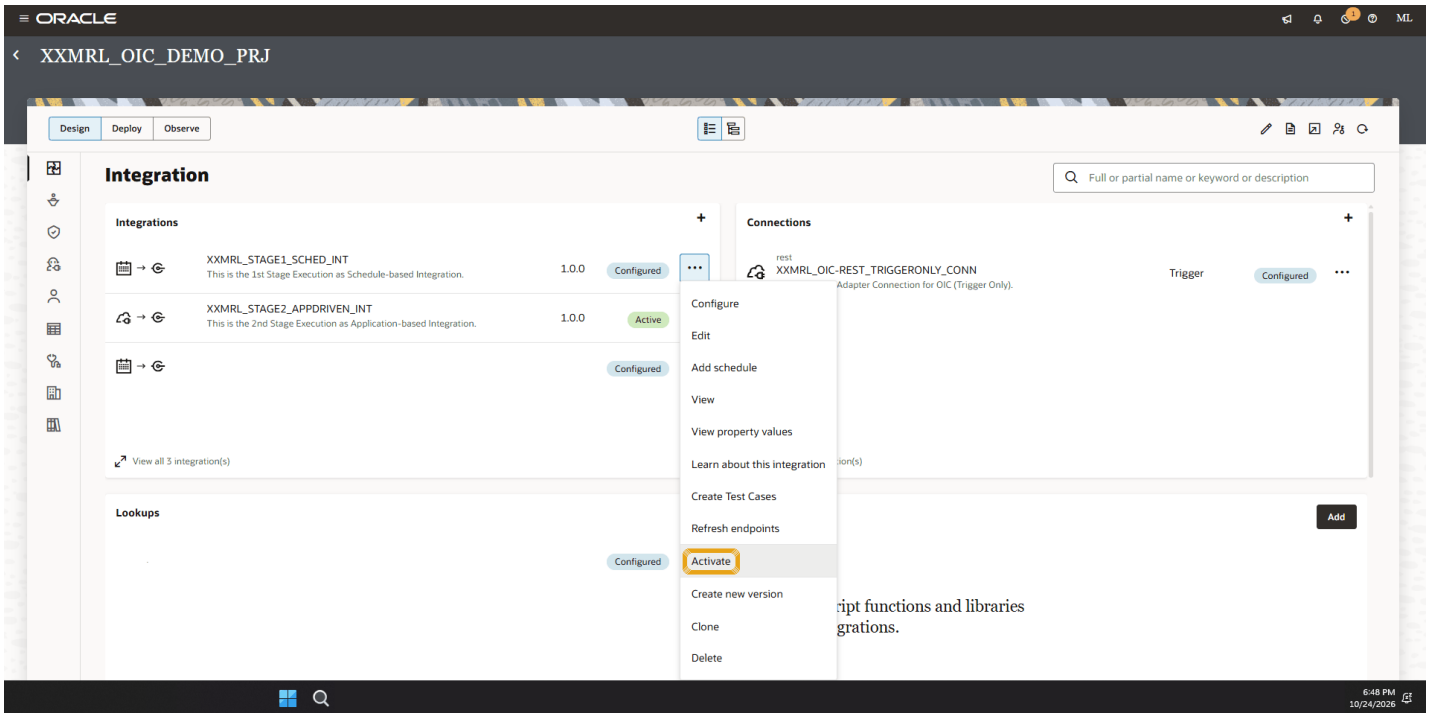
Once done, I clicked Validate on the upper right, waited for the confirmation, and went back to the main integration orchestration:



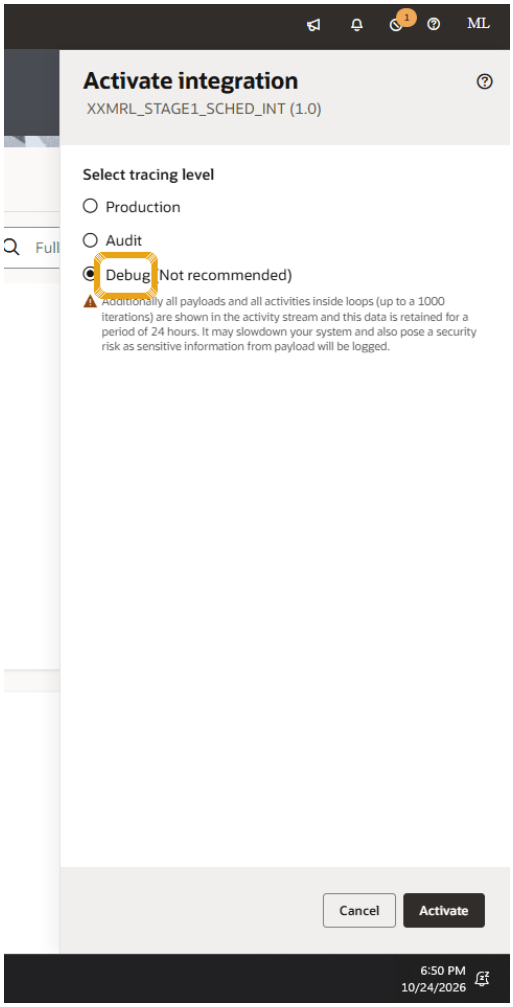
I clicked Save and went back to the project.

Part 3.2: Activate and Test the Schedule Integration

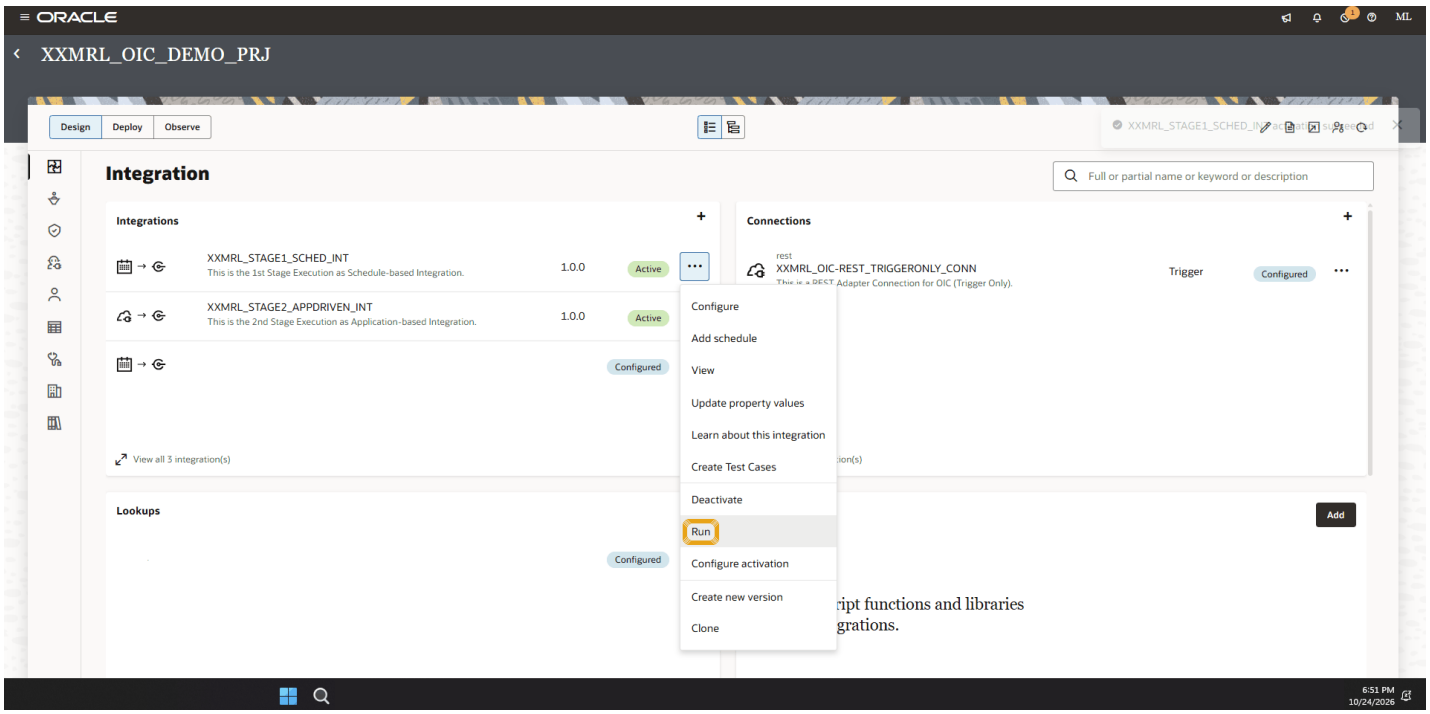
Back in the project, I opened the … menu for the schedule integration and chose Activate:



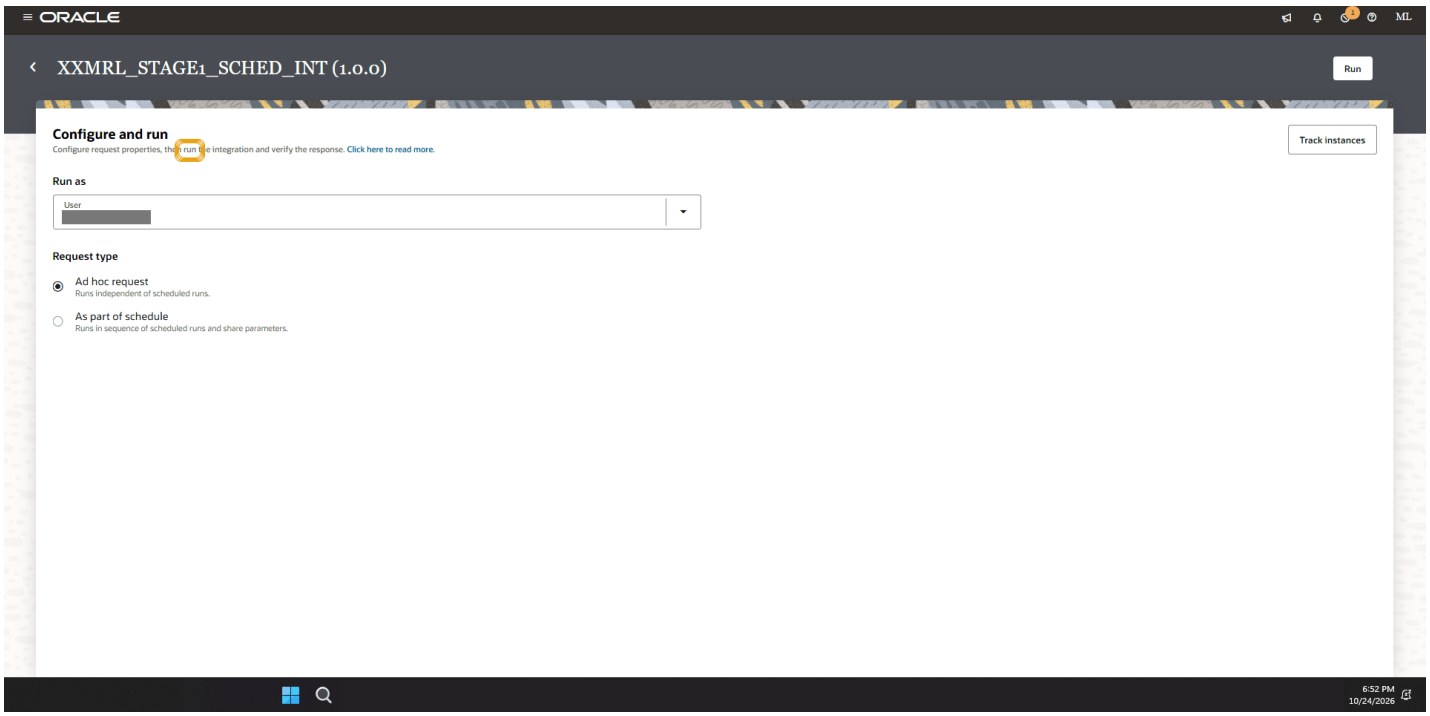
I selected the Debug tracing level and clicked Activate:



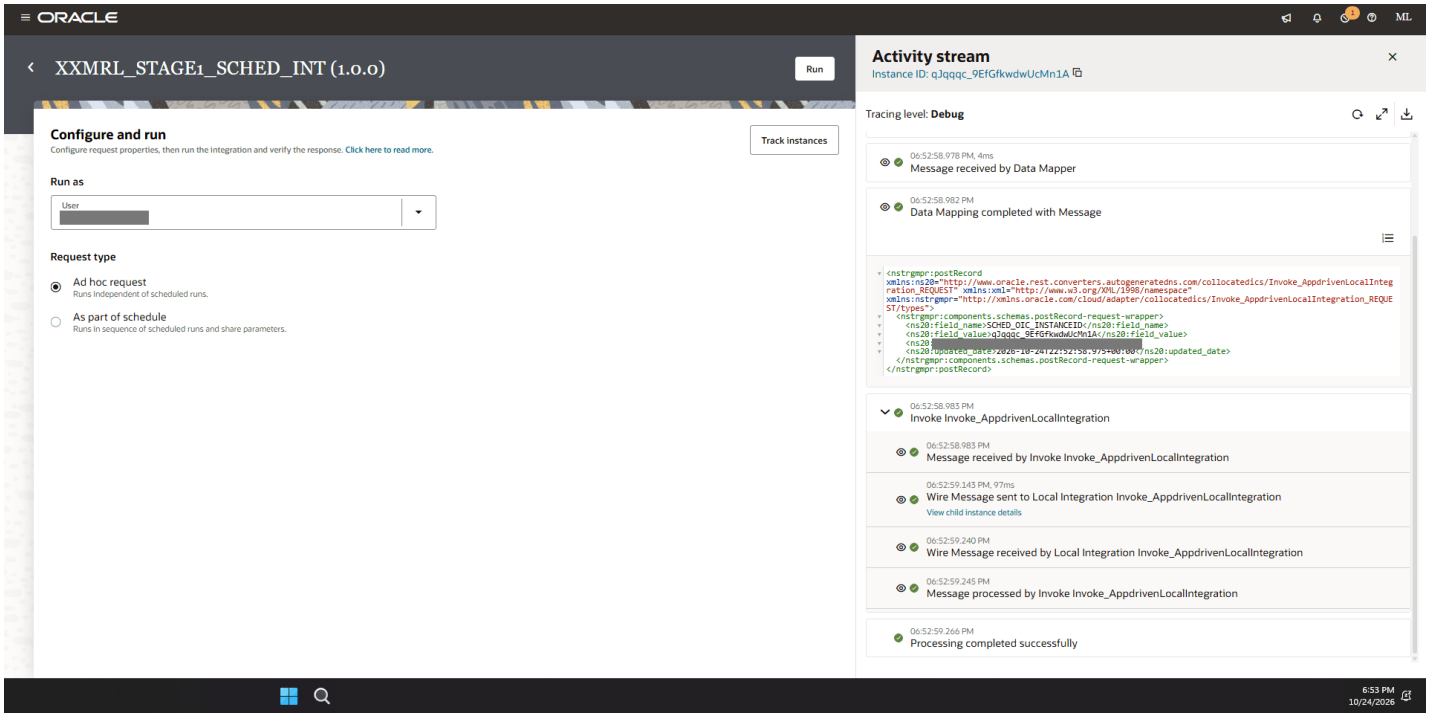
Once it was active, I opened the … menu and chose Run:



I kept the default values (Ad hoc request) and clicked Run:



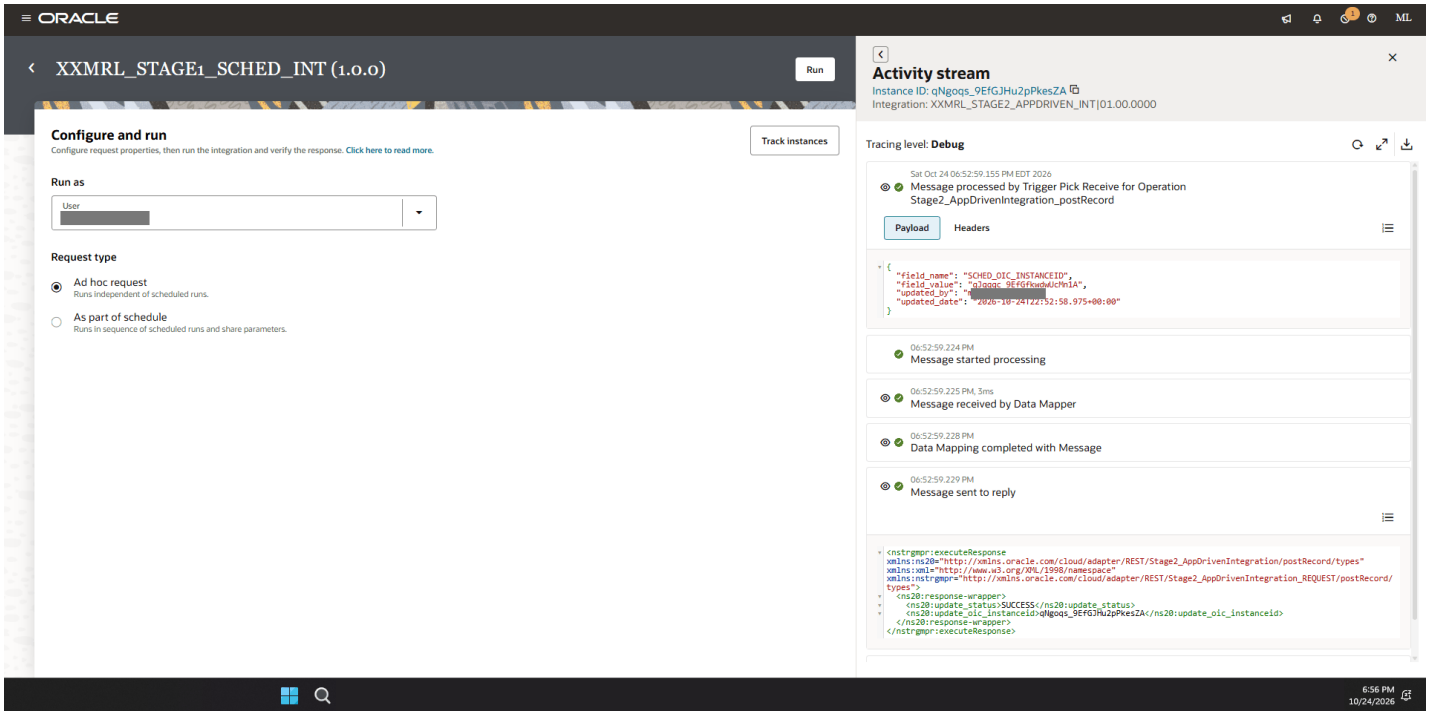
The activity stream opened, and I expanded the audit entries to see more detail:



These are the values the schedule integration mapped into the request payload for the app-driven integration:

```
<nstrgmpr:components.schemas.postRecord-request-wrapper>
<ns20:field_name>SCHED_OIC_INSTANCEID</ns20:field_name>
<ns20:field_value>qJqqqc_9EfGfkwdwUcMn1A</ns20:field_value>
<ns20:updated_by>[user]</ns20:updated_by>
<ns20:updated_date>2026-10-24T22:52:58.975+00:00</ns20:updated_date>
</nstrgmpr:components.schemas.postRecord-request-wrapper>
```

On the Invoke_AppdrivenLocalIntegration step, I clicked View child instance details to open the child instance, which is the app-driven integration run:

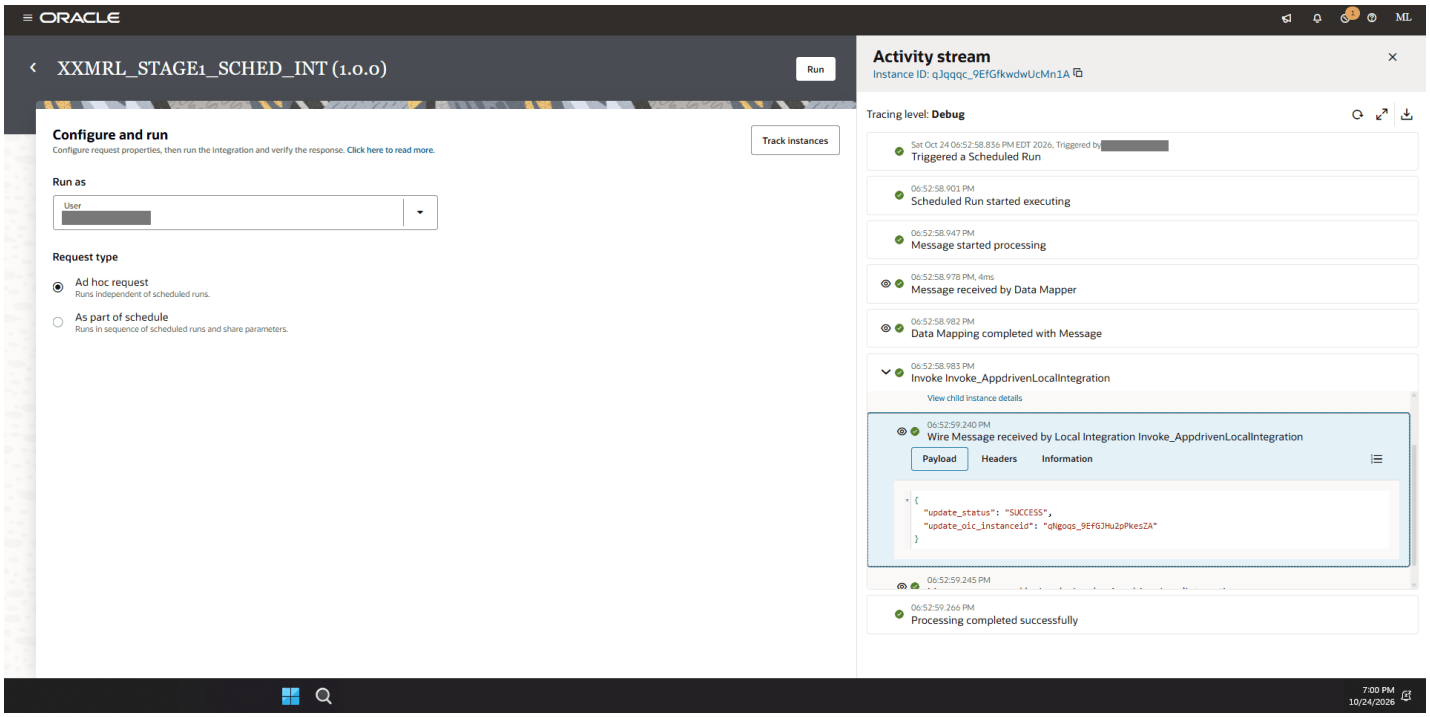


Expanding the child activity stream confirmed that the app-driven integration received the values sent by the schedule integration and returned a response:

```
<nstrgmpr:executeResponse
xmlns:ns20="http://xmlns.oracle.com/cloud/adapter/REST/Stage2_AppDrivenIntegration/postRecord/types"
xmlns:xml="http://www.w3.org/XML/1998/namespace"
xmlns:nstrgmpr="http://xmlns.oracle.com/cloud/adapter/REST/Stage2_AppDrivenIntegration_REQUEST/postRecord/types"
>
<ns20:response-wrapper>
<ns20:update_status>SUCCESS</ns20:update_status>
<ns20:update_oic_instanceid>qNgoqs_9EfGJHu2pPkesZA</ns20:update_oic_instanceid>
</ns20:response-wrapper>
</nstrgmpr:executeResponse>
```

Back in the schedule integration's activity stream, I expanded Wire Message received by Local Integration Invoke_AppdrivenLocalIntegration. The payload matched the app-driven integration's response:

```
{
"update_status": "SUCCESS",
"update_oic_instanceid": "qNgoqs_9EfGJHu2pPkesZA"
}
```



Mapping Summary

The mapper expressions used in the three mappings, taken from the exported project:

Mapping	Target field	Expression
XXMRL_STAGE2_APPDRIVEN_INT, getRecord response	`get_status`	`'SUCCESS'`
	`get_oic_instanceid`	`$self/nsmpr0:metadata/nsmpr0:runtime/nsmpr0:instanceId`
XXMRL_STAGE2_APPDRIVEN_INT, postRecord response	`update_status`	`'SUCCESS'`
	`update_oic_instanceid`	`$self/nsmpr0:metadata/nsmpr0:runtime/nsmpr0:instanceId`
XXMRL_STAGE1_SCHED_INT, postRecord request	`field_name`	`'SCHED_OIC_INSTANCEID'`
	`field_value`	`$self/nsmpr0:metadata/nsmpr0:runtime/nsmpr0:instanceId`
	`updated_by`	`$self/nsmpr0:metadata/nsmpr0:runtime/nsmpr0:invokedBy`
	`updated_date`	`/nssrcmpr:schedule/nssrcmpr:startTime`